

Introduction to Self-stabilization

Franck Petit

(with some materials from M. Herlihy & S. Tixeuil)

Sorbonne University

Firstname.Lastname@lip6.fr

Distributed Systems

- ▶ Locality of information
- ▶ Locality of time
- ▶ $\Rightarrow$ **non-determinism**

Definition (Configuration)

Product of the local states of the system components.

Definition (Execution)

Interleaving of the local executions of the system components.

Fault Tolerance

Definition ((Distributed) Task)

A task is **specified** in terms of:

- ▶ **Safety:** *Bad actions*, which should not happen
 - ▶ At the intersection, traffic lights are green on two different axes.
 - ▶ Processes enter the critical section simultaneously.
 - ▶ *Windows* crashes.

Fault Tolerance

Definition ((Distributed) Task)

A task is **specified** in terms of:

- ▶ **Safety:** *Bad actions*, which should not happen
- ▶ **Liveness:** *Good actions*, which should (eventually) happen
 - ▶ At the intersection, if one of the traffic lights is red then, it eventually becomes green.
 - ▶ Every process eventually enter the critical section.
 - ▶ *Windows* eventually reboots.

Fault Tolerance

Definition (Fault)

A fault is an action that corrupts the task specification by changing the correct functioning of a system component.

- ▶ At the intersection, traffic lights are off.
- ▶ A process requesting the critical section is stuck.
- ▶ *Windows* boot loops on a blue screen with white markings.

Fault Tolerance

Definition (Fault)

A fault is an action that corrupts the task specification by changing the correct functioning of a system component.

- ▶ Type $\rightarrow$ fail-stop, crash, omission, Byzantine, ...
- ▶ Duration
- ▶ Detection Rate
- ▶ Correction Rate
- ▶ Frequency

Fault-tolerant algorithm $\Rightarrow$ Tolerates a given class of faults

Fault Tolerance

- ▶ **Masking FT:** Both *Safety* and *Liveness* must be guaranteed. Unfortunately, **[FLP]!**
- ▶ **Fail-Safe FT:** *Safety* guaranteed but not *Liveness*.
- ▶ **Non-Masking FT:** *Liveness* guaranteed but not *Safety*.
Self-Stabilization: *Safety* **eventually** guaranteed.

Self-stabilization

Definition (Self-stabilizing System)

Starting from **any** initial configuration, the system **eventually** reaches a configuration from which its behavior is correct.

- ▶ Defined by Dijkstra in 1974
- ▶ Advocated by Lamport in 1984 to address fault-tolerant issues

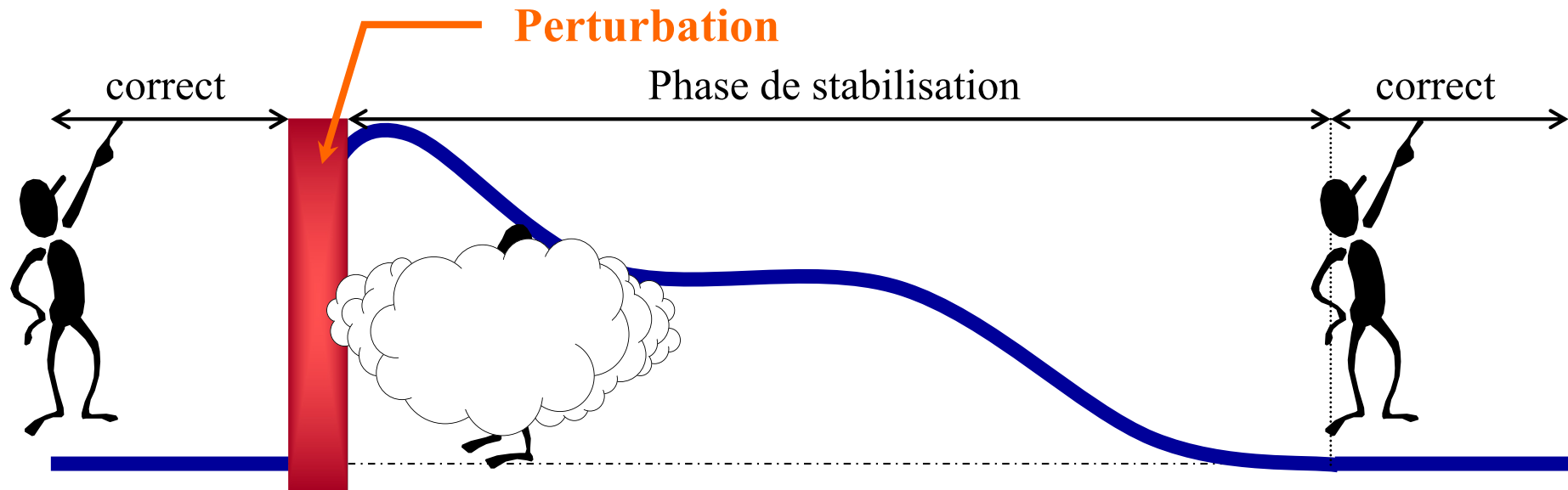
■ Auto-stabilisation

- ① Messages incorrects dans les canaux
- ② Valeurs incorrectes dans les variables

Quel que soit son état initial, le système est sûr d'atteindre par lui-même un état légitime en un nombre fini d'étapes.

Dijkstra 1974

■ Auto-stabilisation

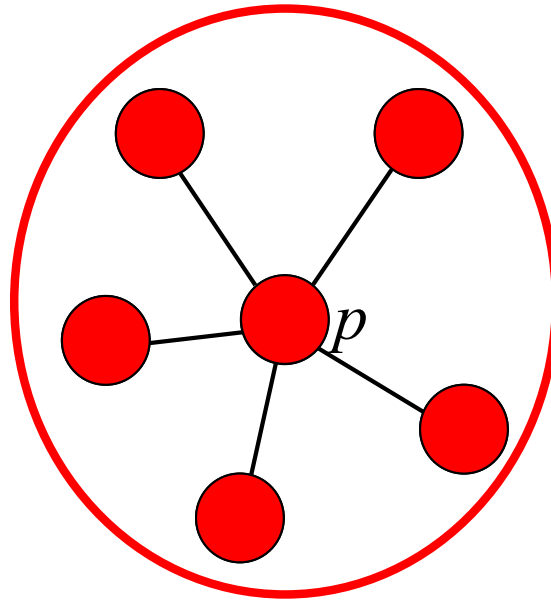


Quel que soit son état initial, le système est sûr d'atteindre par lui-même un état légitime en un nombre fini d'étapes.

Dijkstra 1974

Modèle à états

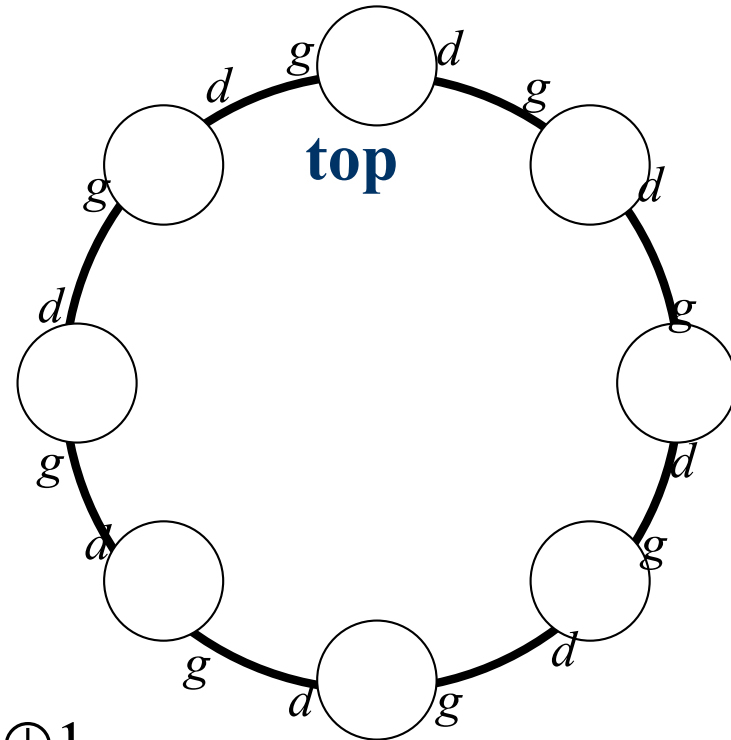
<Garde> → <Action>



Anneau à jeton

Spécification

- Sûreté : *Au plus un jeton dans le système*
- Vivacité : *Chaque processeur obtient le jeton infiniment souvent*



$p = top$

TR1 : $(v_p = v_g) \rightarrow v_p := v_g \oplus 1$

$p \neq top$

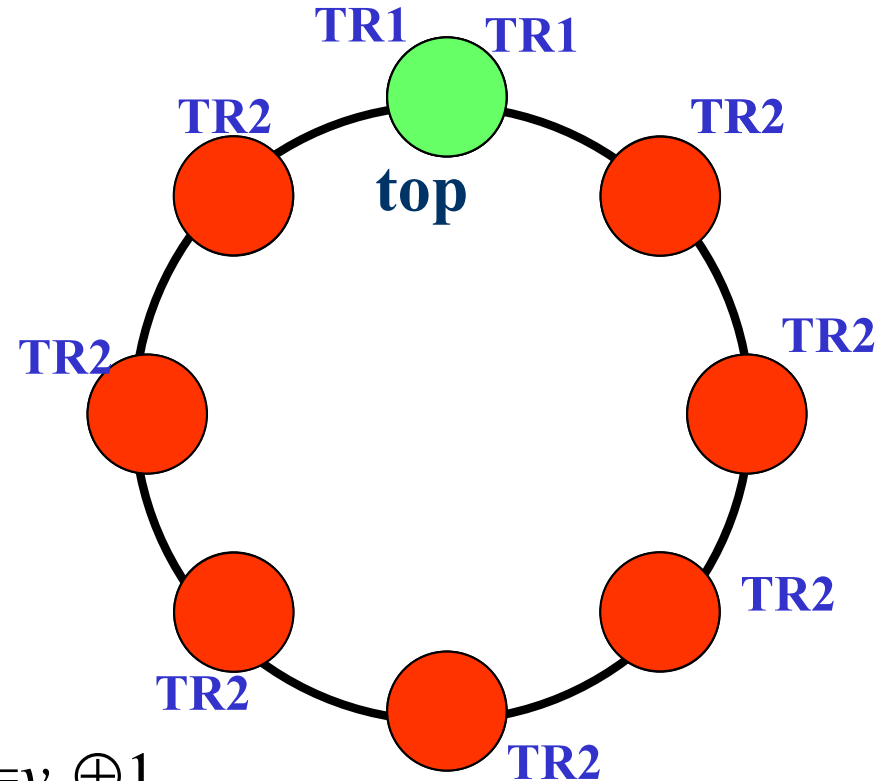
TR2 : $(v_p \neq v_g) \rightarrow v_p := v_g$

Anneau à jeton

Auto-stabilisant ?

Spécification

- Sûreté : *Au plus un jeton dans le système*
- Vivacité : *Chaque processeur obtient le jeton infiniment souvent*



$p = top$

TR1 : $(v_p = v_g) \rightarrow v_p := v_g \oplus 1$

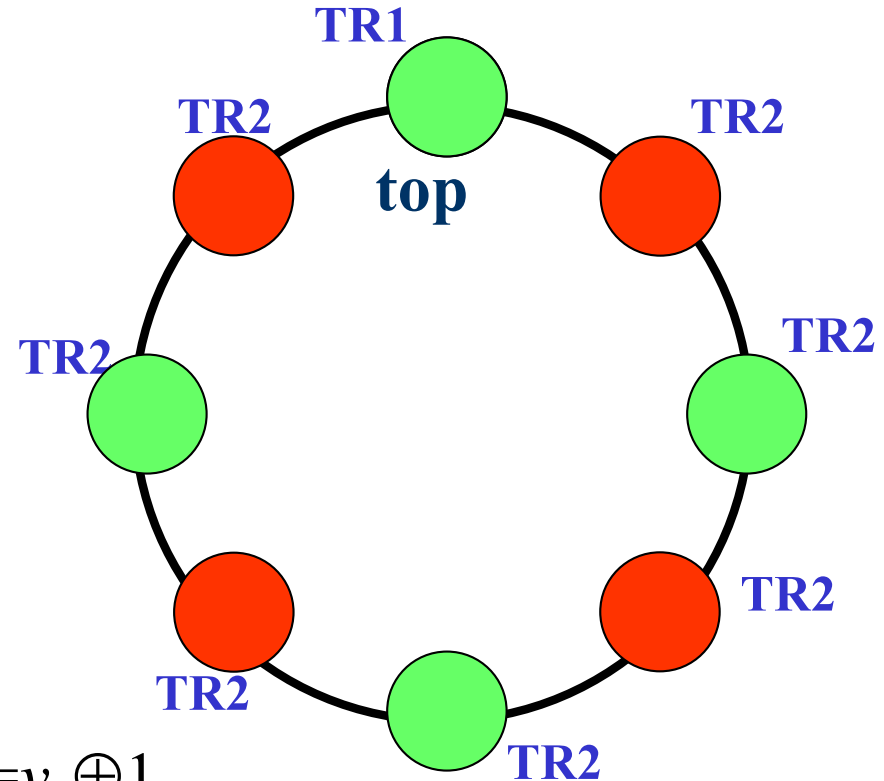
$p \neq top$

TR2 : $(v_p \neq v_g) \rightarrow v_p := v_g$

Anneau à jeton

Spécification

- Sûreté : *Au plus un jeton dans le système*
- Vivacité : *Chaque processeur obtient le jeton infiniment souvent*



$p = top$

TR1 : $(v_p = v_g) \rightarrow v_p := v_g \oplus 1$

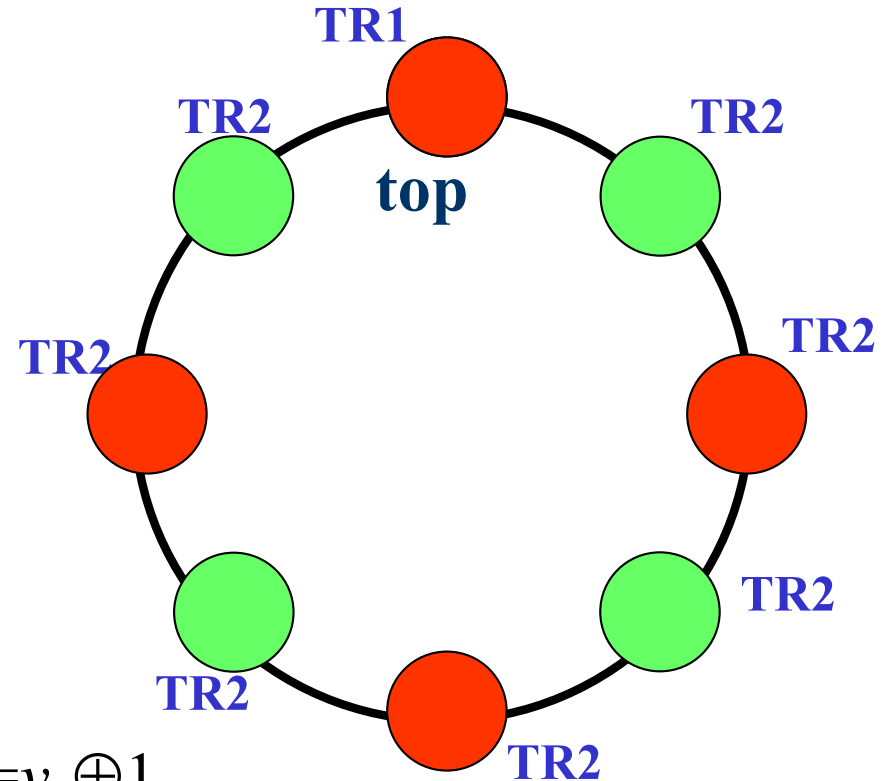
$p \neq top$

TR2 : $(v_p \neq v_g) \rightarrow v_p := v_g$

Anneau à jeton

Spécification

- Sûreté : *Au plus un jeton dans le système*
- Vivacité : *Chaque processeur obtient le jeton infiniment souvent*



$p = top$

$TR1 : (v_p = v_g) \rightarrow v_p := v_g \oplus 1$

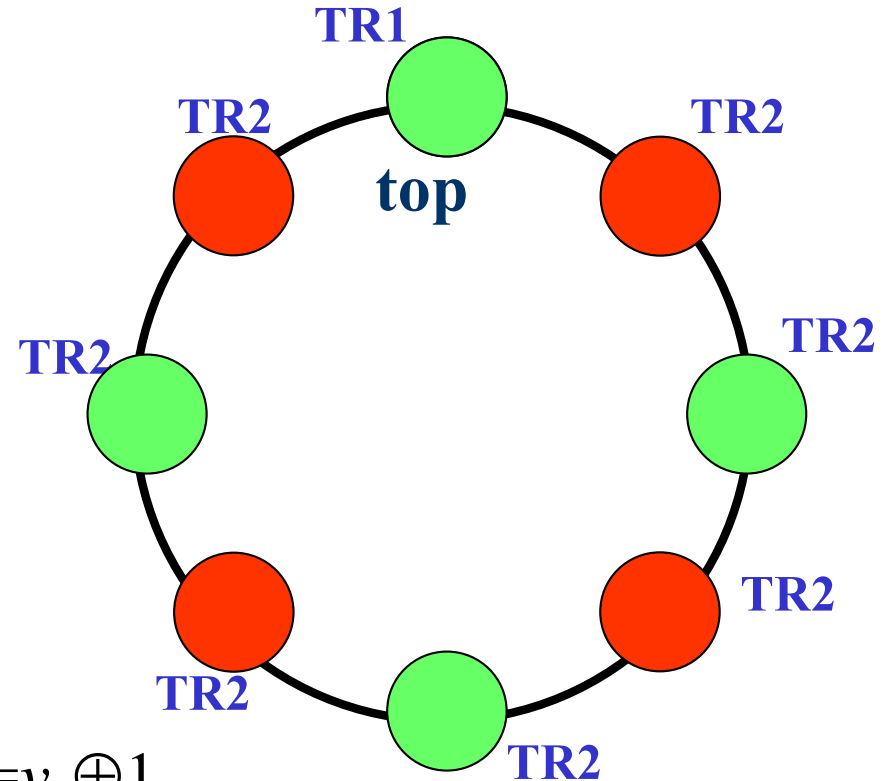
$p \neq top$

$TR2 : (v_p \neq v_g) \rightarrow v_p := v_g$

Anneau à jeton

Spécification

- Sûreté : *Au plus un jeton dans le système*
- Vivacité : *Chaque processeur obtient le jeton infiniment souvent*



$p = top$

TR1 : $(v_p = v_g) \rightarrow v_p := v_g \oplus 1$

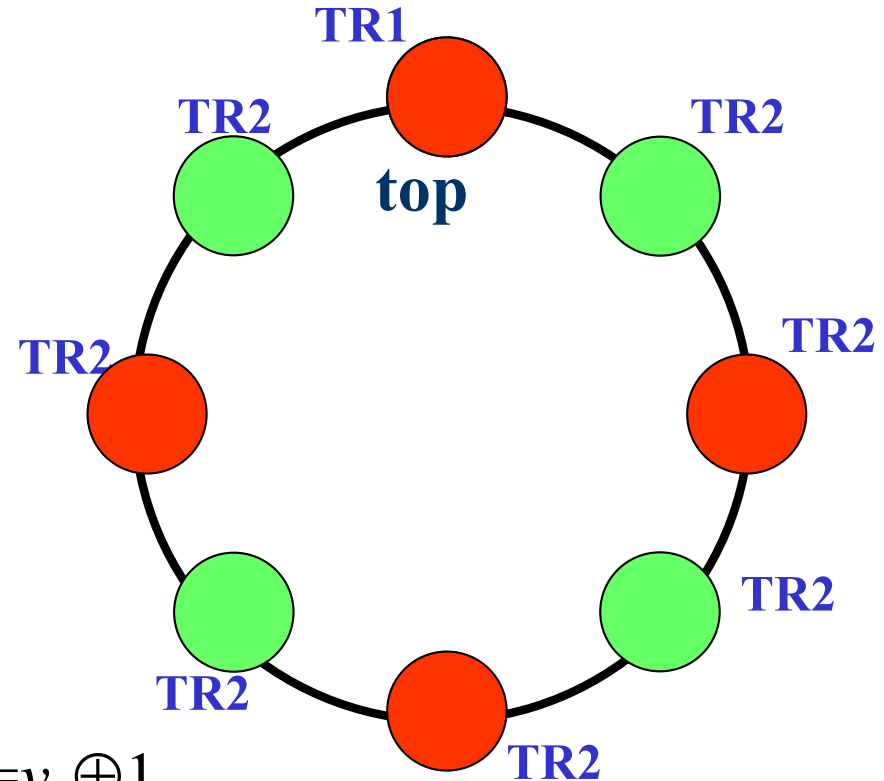
$p \neq top$

TR2 : $(v_p \neq v_g) \rightarrow v_p := v_g$

Anneau à jeton

Spécification

- Sûreté : *Au plus un jeton dans le système*
- Vivacité : *Chaque processeur obtient le jeton infiniment souvent*



$p = top$

TR1 : $(v_p = v_g) \rightarrow v_p := v_g \oplus 1$

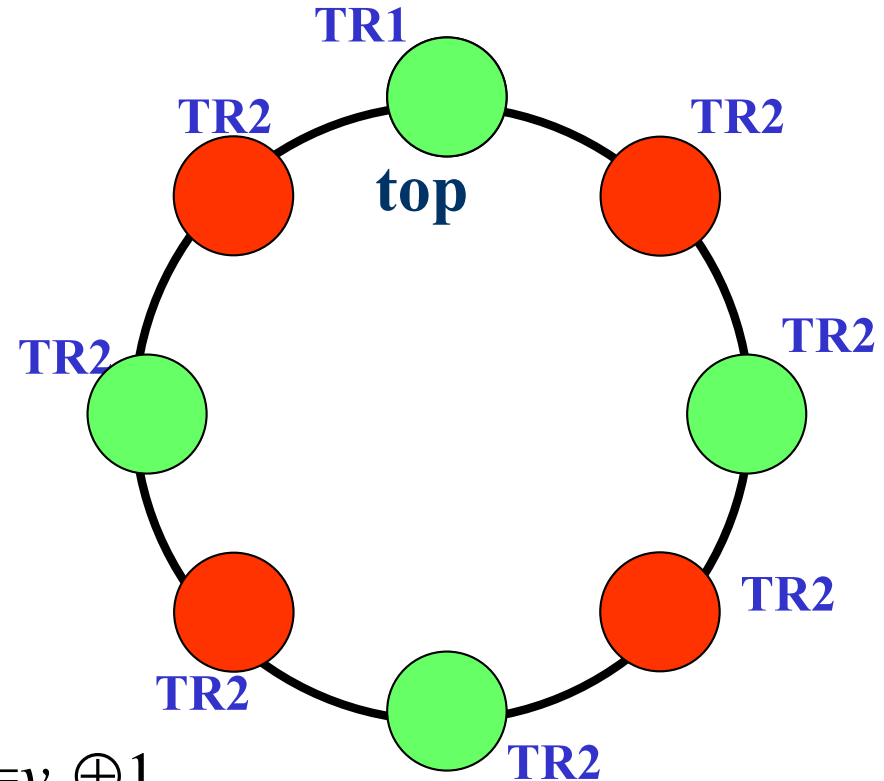
$p \neq top$

TR2 : $(v_p \neq v_g) \rightarrow v_p := v_g$

Anneau à jeton

Spécification

- Sûreté : *Au plus un jeton dans le système*
- Vivacité : *Chaque processeur obtient le jeton infiniment souvent*



$p = top$

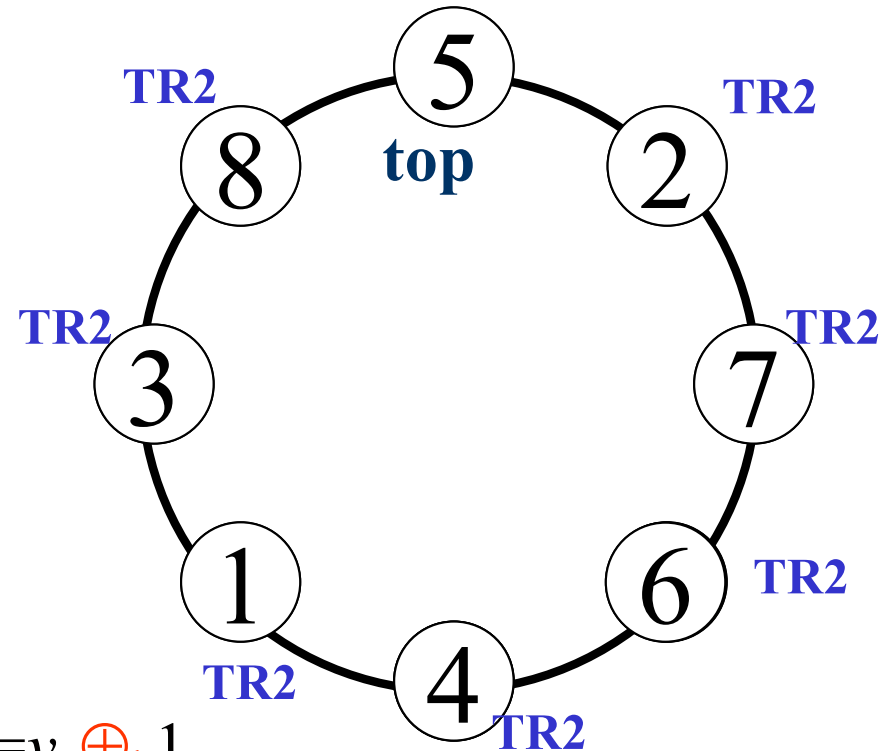
TR1 : $(v_p = v_g) \rightarrow v_p := v_g \oplus 1$

$p \neq top$

TR2 : $(v_p \neq v_g) \rightarrow v_p := v_g$

Anneau à jeton

Algorithme Auto-stabilisant de circulation de jeton de Dijkstra



$p = top$

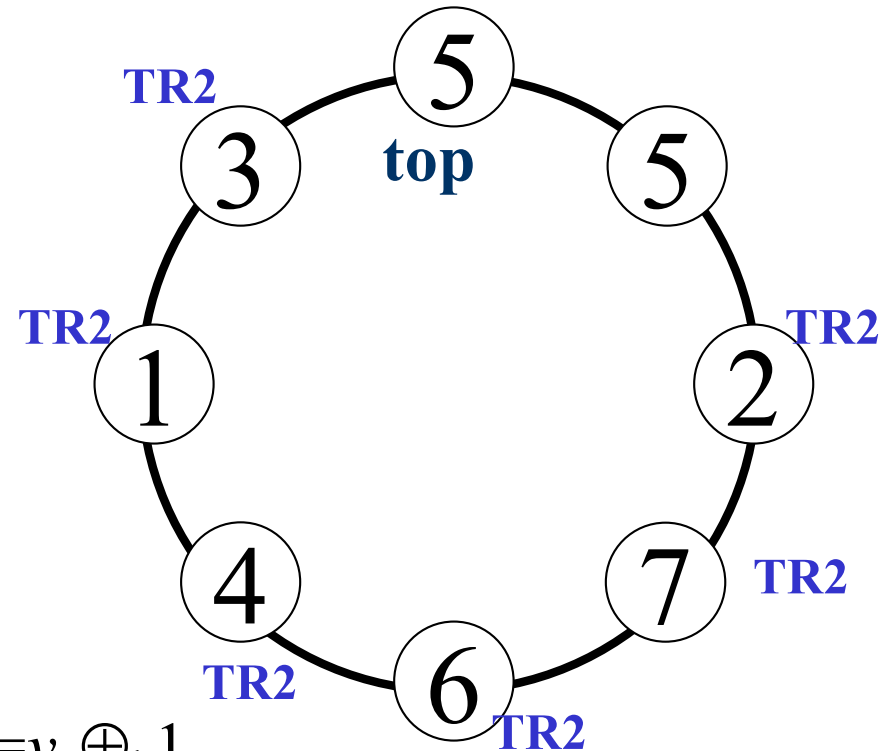
TR1 : $(v_p = v_g)$ $\rightarrow$ $v_p := v_g \oplus_k 1$

$p \neq top$

TR2 : $(v_p \neq v_g)$ $\rightarrow$ $v_p := v_g$

Anneau à jeton

Algorithme Auto-stabilisant de circulation de jeton de Dijkstra



$p = top$

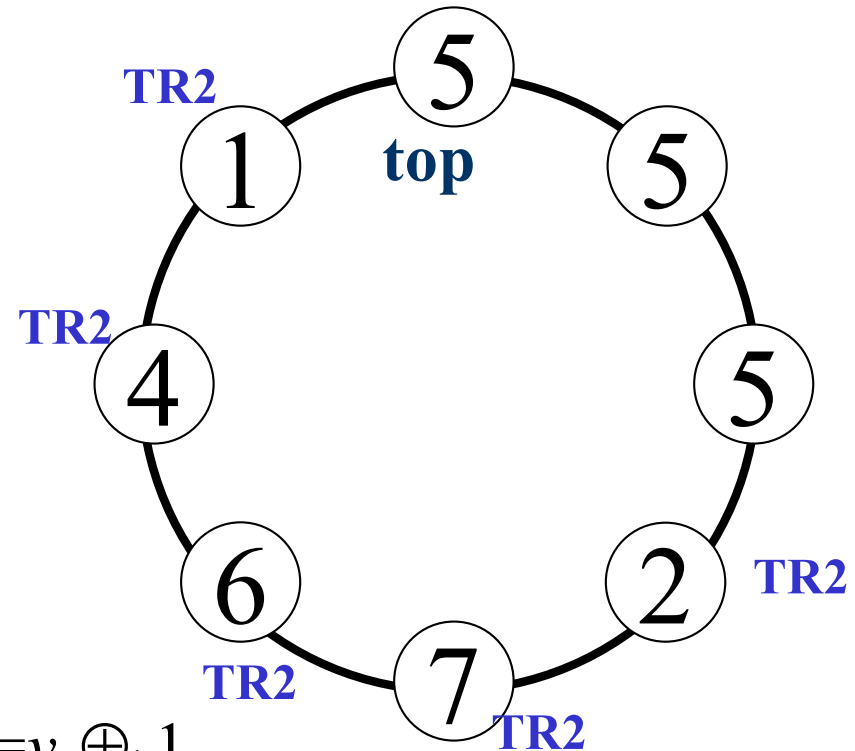
TR1 : $(v_p = v_g)$ $\rightarrow$ $v_p := v_g \oplus_k 1$

$p \neq top$

TR2 : $(v_p \neq v_g)$ $\rightarrow$ $v_p := v_g$

Anneau à jeton

Algorithme Auto-stabilisant de circulation de jeton de Dijkstra



$p = top$

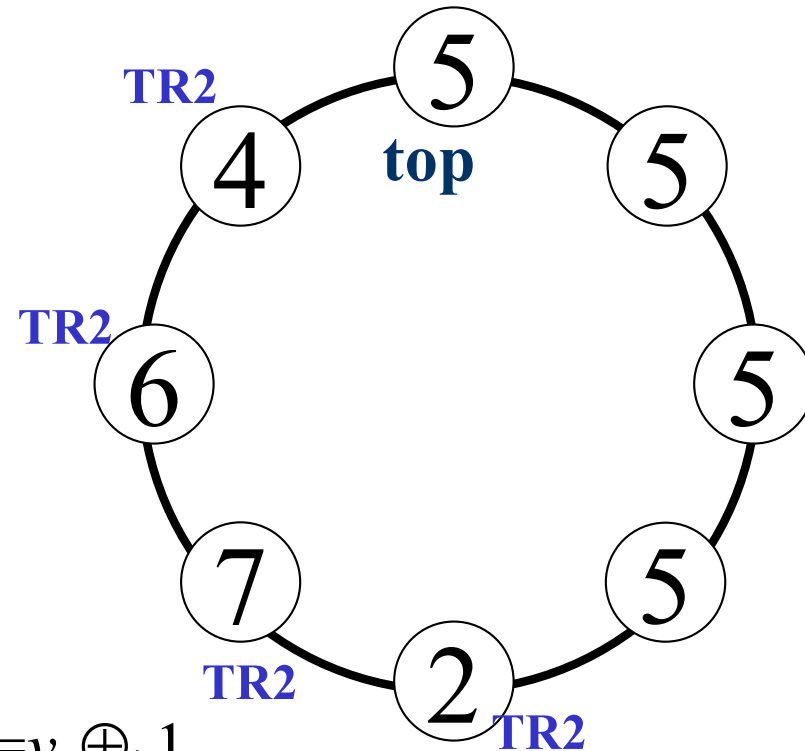
TR1 : $(v_p = v_g)$ $\rightarrow$ $v_p := v_g \oplus_k 1$

$p \neq top$

TR2 : $(v_p \neq v_g)$ $\rightarrow$ $v_p := v_g$

Anneau à jeton

Algorithme Auto-stabilisant de circulation de jeton de Dijkstra



$p = top$

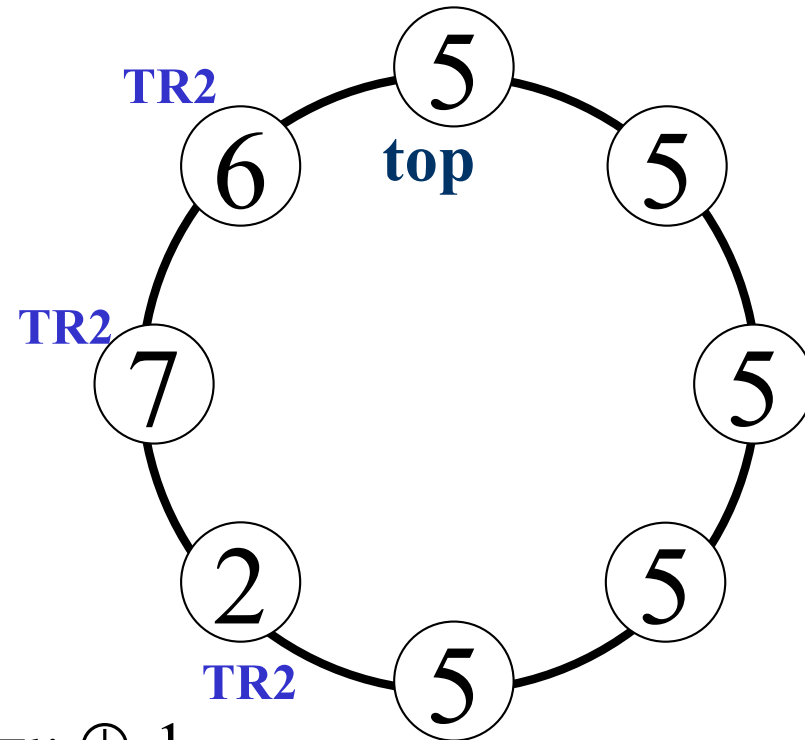
TR1 : $(v_p = v_g)$ $\rightarrow$ $v_p := v_g \oplus_k 1$

$p \neq top$

TR2 : $(v_p \neq v_g)$ $\rightarrow$ $v_p := v_g$

Anneau à jeton

Algorithme Auto-stabilisant de circulation de jeton de Dijkstra



$p = top$

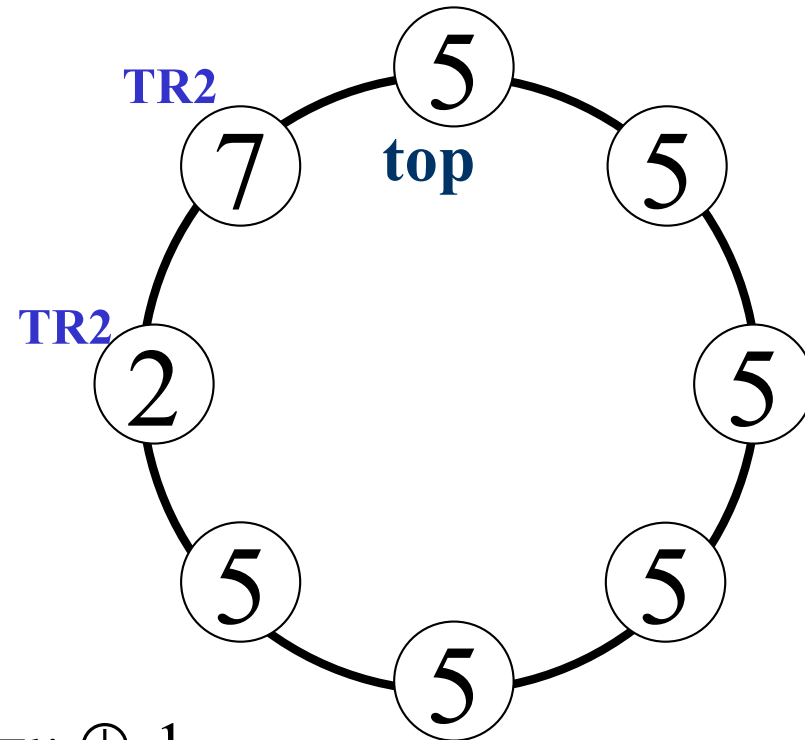
TR1 : $(v_p = v_g)$ $\rightarrow$ $v_p := v_g \oplus_k 1$

$p \neq top$

TR2 : $(v_p \neq v_g)$ $\rightarrow$ $v_p := v_g$

Anneau à jeton

Algorithme Auto-stabilisant de circulation de jeton de Dijkstra



$p = top$

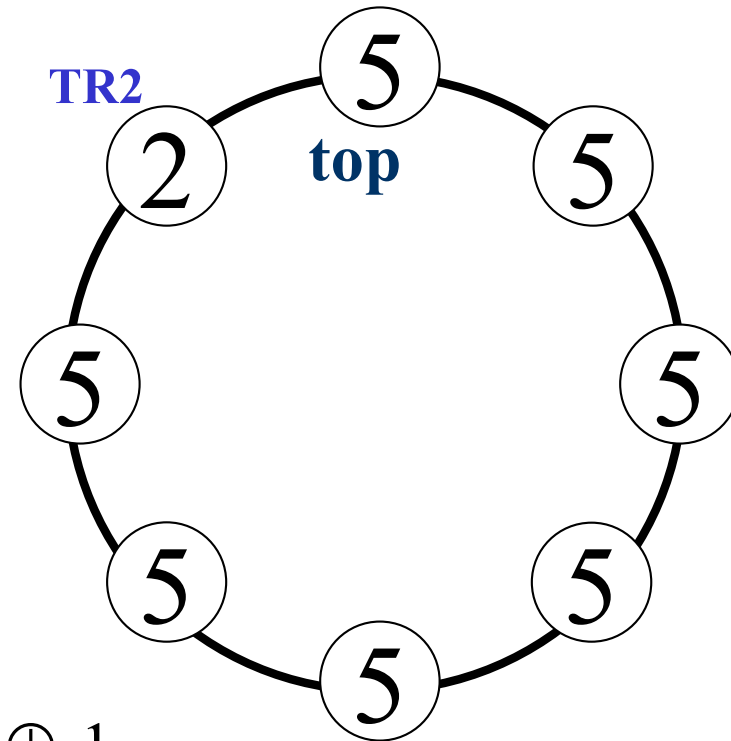
TR1 : $(v_p = v_g)$ $\rightarrow$ $v_p := v_g \oplus_k 1$

$p \neq top$

TR2 : $(v_p \neq v_g)$ $\rightarrow$ $v_p := v_g$

Anneau à jeton

Algorithme Auto-stabilisant de circulation de jeton de Dijkstra



$p = top$

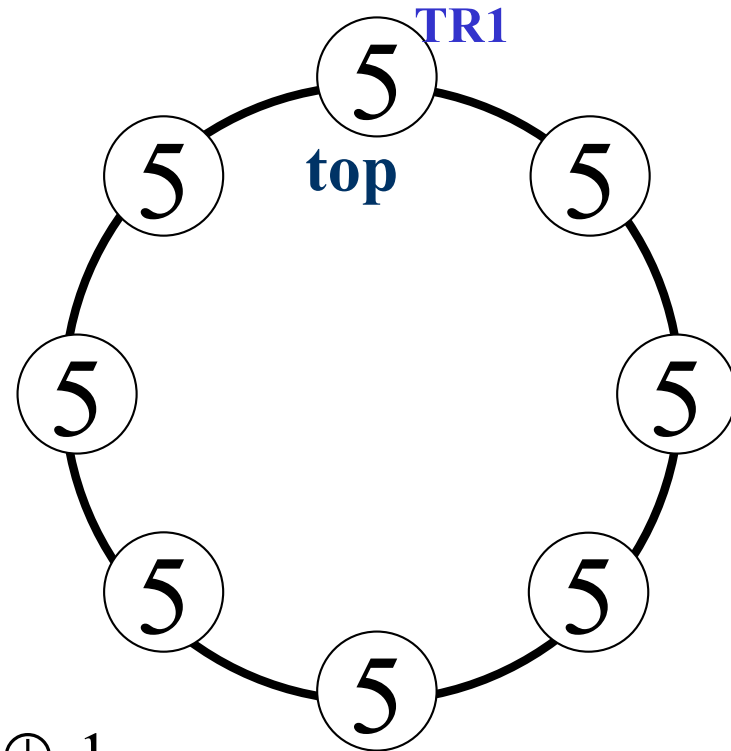
TR1 : $(v_p = v_g)$ $\rightarrow$ $v_p := v_g \oplus_k 1$

$p \neq top$

TR2 : $(v_p \neq v_g)$ $\rightarrow$ $v_p := v_g$

Anneau à jeton

Algorithme Auto-stabilisant de circulation de jeton de Dijkstra



$p = top$

TR1 : $(v_p = v_g)$ $\rightarrow v_p := v_g \oplus_k 1$

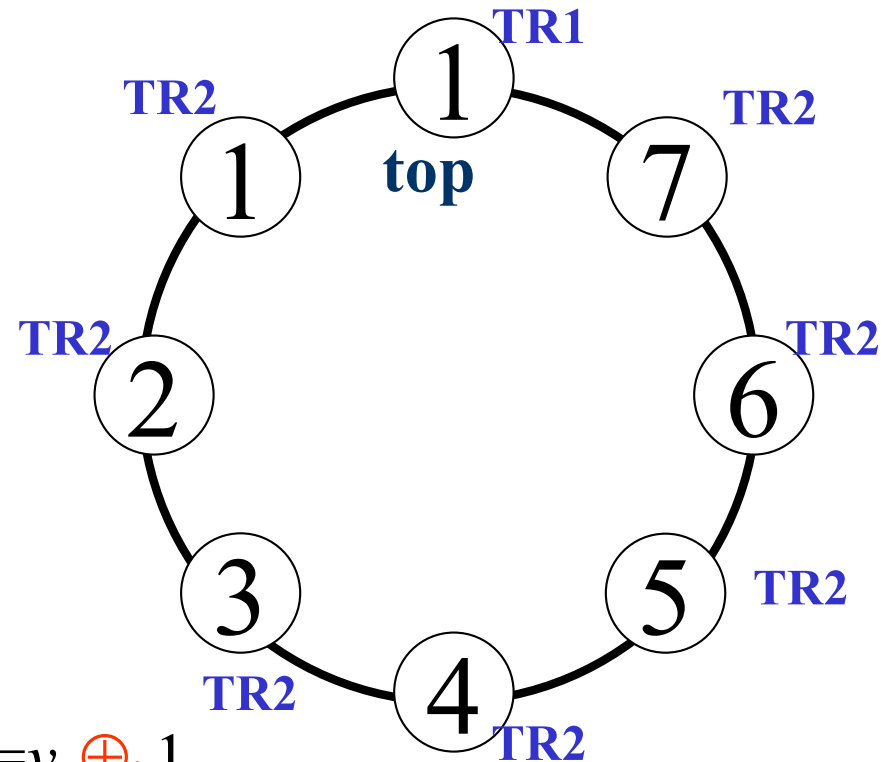
$p \neq top$

TR2 : $(v_p \neq v_g)$ $\rightarrow v_p := v_g$

Valeur de k ?

Anneau à jeton

Algorithme Auto-stabilisant de circulation de jeton de Dijkstra



$p = top$

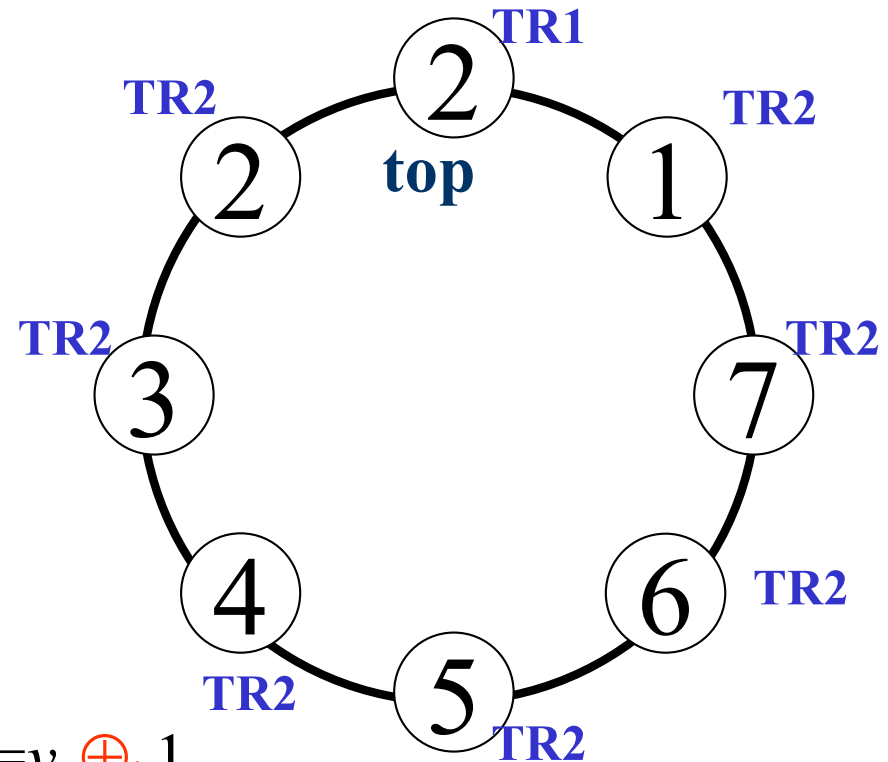
TR1 : $(v_p = v_g)$ $\rightarrow$ $v_p := v_g \oplus_k 1$

$p \neq top$

TR2 : $(v_p \neq v_g)$ $\rightarrow$ $v_p := v_g$

Anneau à jeton

Algorithme Auto-stabilisant de circulation de jeton de Dijkstra



$p = top$

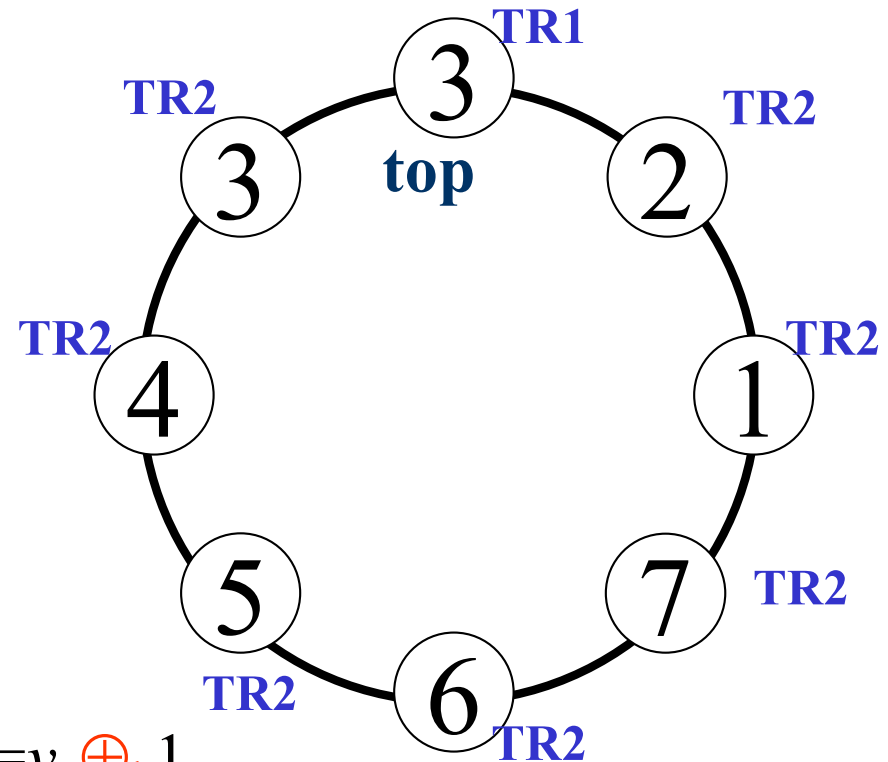
TR1 : $(v_p = v_g)$ $\rightarrow v_p := v_g \oplus_k 1$

$p \neq top$

TR2 : $(v_p \neq v_g)$ $\rightarrow v_p := v_g$

Anneau à jeton

Algorithme Auto-stabilisant de circulation de jeton de Dijkstra



$p = top$

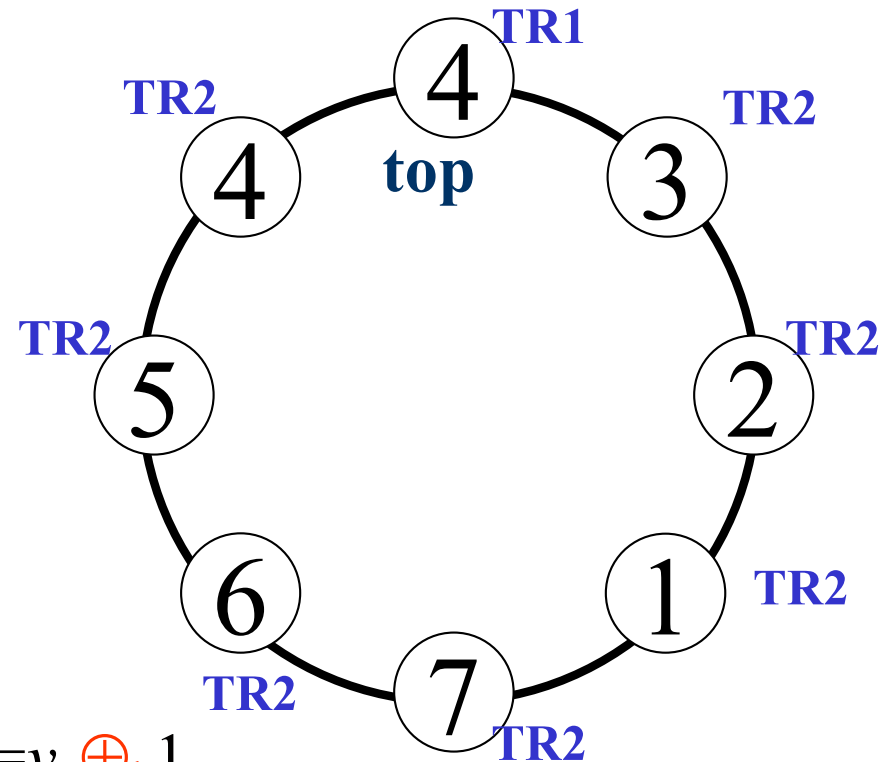
TR1 : $(v_p = v_g)$ $\rightarrow v_p := v_g \oplus_k 1$

$p \neq top$

TR2 : $(v_p \neq v_g)$ $\rightarrow v_p := v_g$

Anneau à jeton

Algorithme Auto-stabilisant de circulation de jeton de Dijkstra



$p = top$

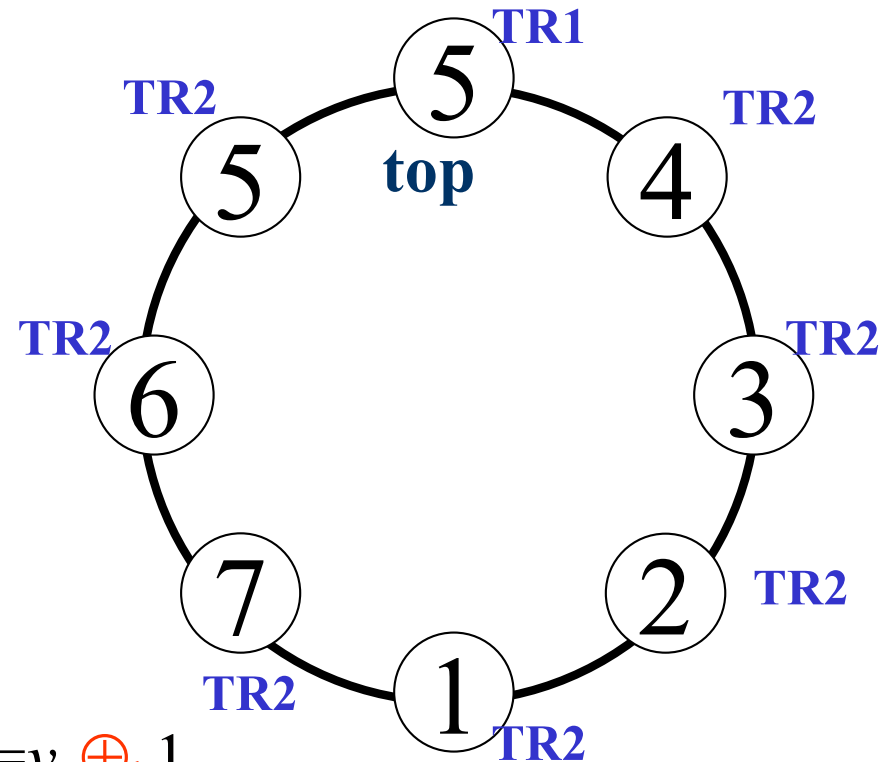
TR1 : $(v_p = v_g)$ $\rightarrow$ $v_p := v_g \oplus_k 1$

$p \neq top$

TR2 : $(v_p \neq v_g)$ $\rightarrow$ $v_p := v_g$

Anneau à jeton

Algorithme Auto-stabilisant de circulation de jeton de Dijkstra



$p = top$

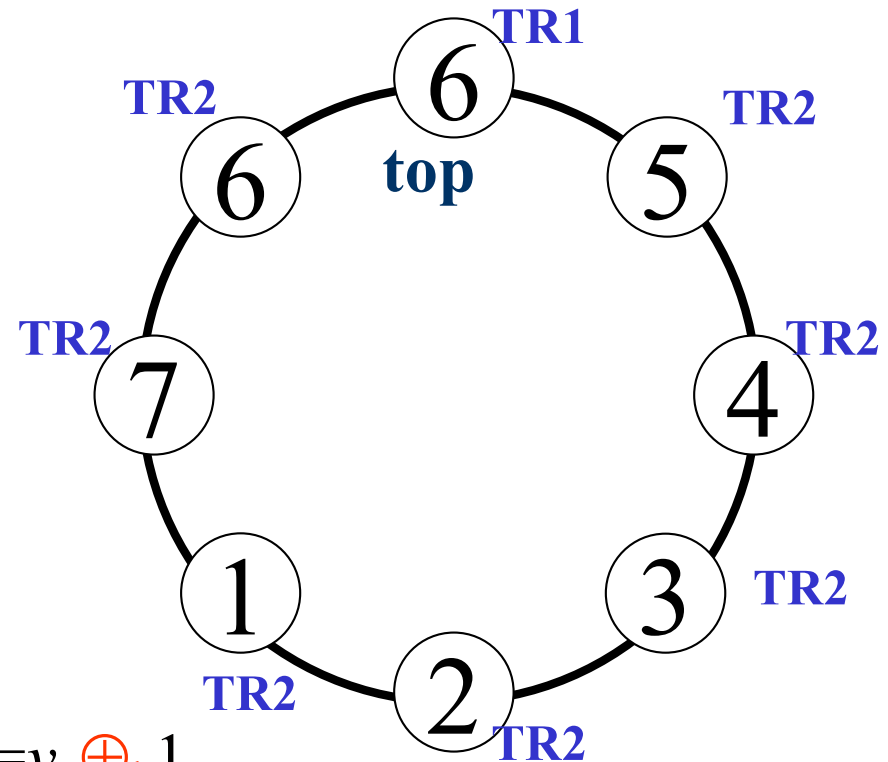
TR1 : $(v_p = v_g)$ $\rightarrow v_p := v_g \oplus_k 1$

$p \neq top$

TR2 : $(v_p \neq v_g)$ $\rightarrow v_p := v_g$

Anneau à jeton

Algorithme Auto-stabilisant de circulation de jeton de Dijkstra



$p = top$

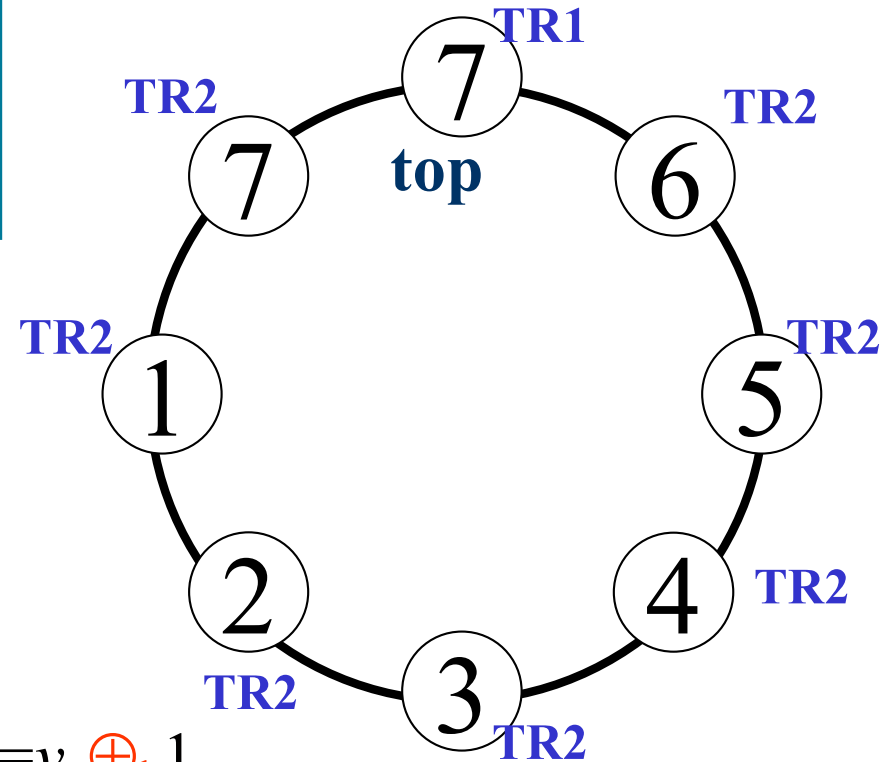
TR1 : $(v_p = v_g)$ $\rightarrow$ $v_p := v_g \oplus_k 1$

$p \neq top$

TR2 : $(v_p \neq v_g)$ $\rightarrow$ $v_p := v_g$

Anneau à jeton

Algorithme Auto-stabilisant de circulation de jeton de Dijkstra



$p = top$

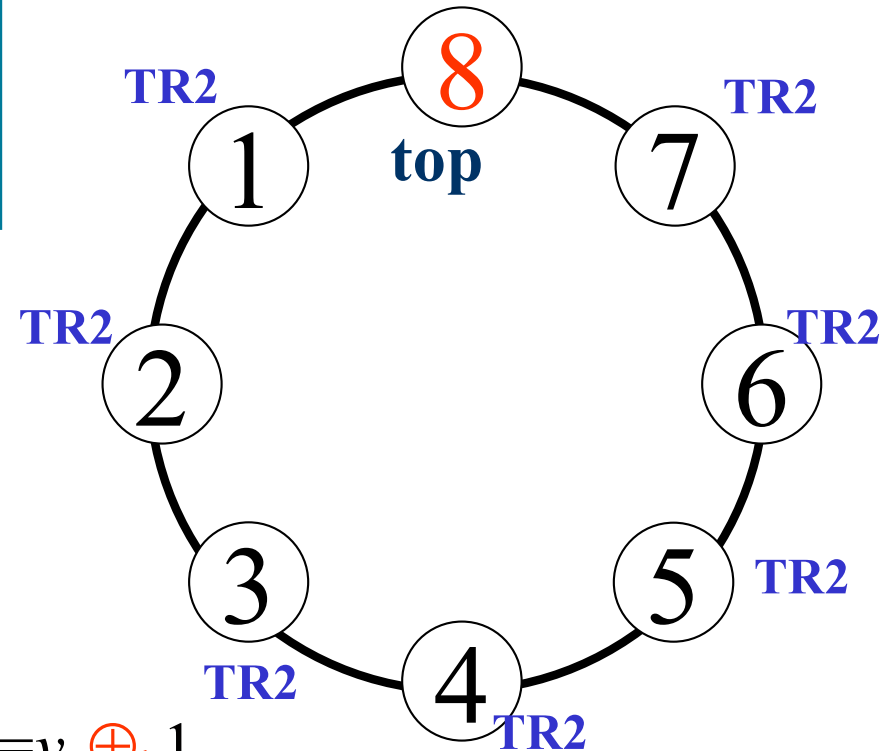
TR1 : $(v_p = v_g)$ $\rightarrow v_p := v_g \oplus_k 1$

$p \neq top$

TR2 : $(v_p \neq v_g)$ $\rightarrow v_p := v_g$

Anneau à jeton

Algorithme Auto-stabilisant de circulation de jeton de Dijkstra



$p = top$

TR1 : $(v_p = v_g)$ $\rightarrow v_p := v_g \oplus_k 1$

$p \neq top$

TR2 : $(v_p \neq v_g)$ $\rightarrow v_p := v_g$

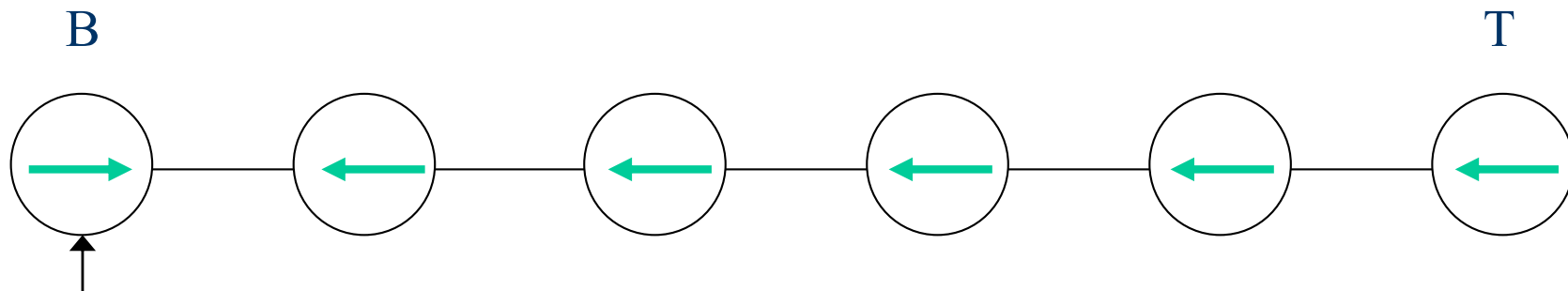
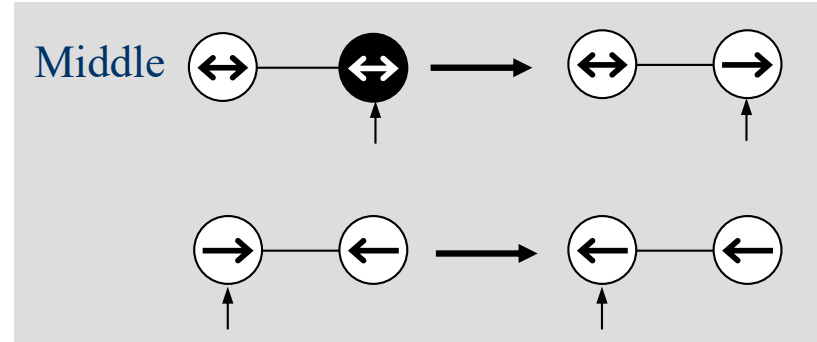
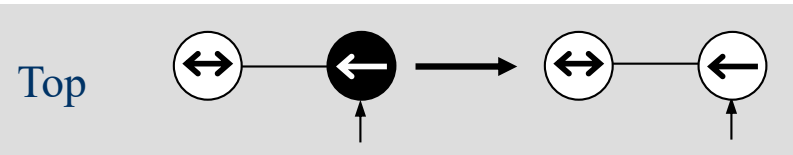
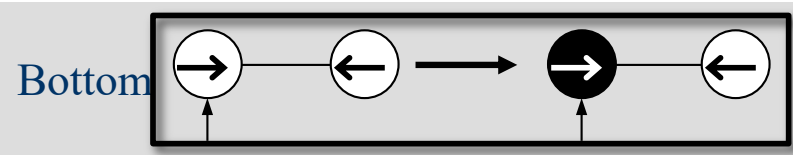
Temps de Stabilisation : $O(n)$

Bottom  

Top  

Middle    

 [Dijkstra 74]



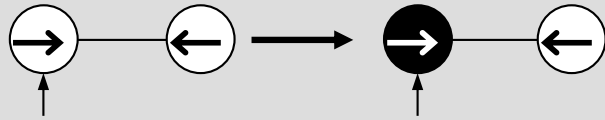
Bottom  

Top  

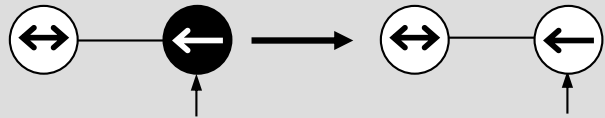
Middle    

 [Dijkstra 74]

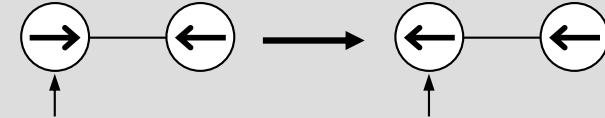
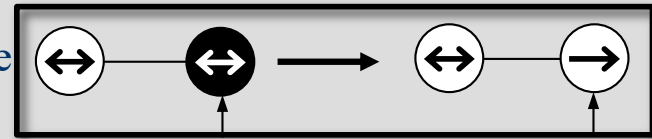
Bottom



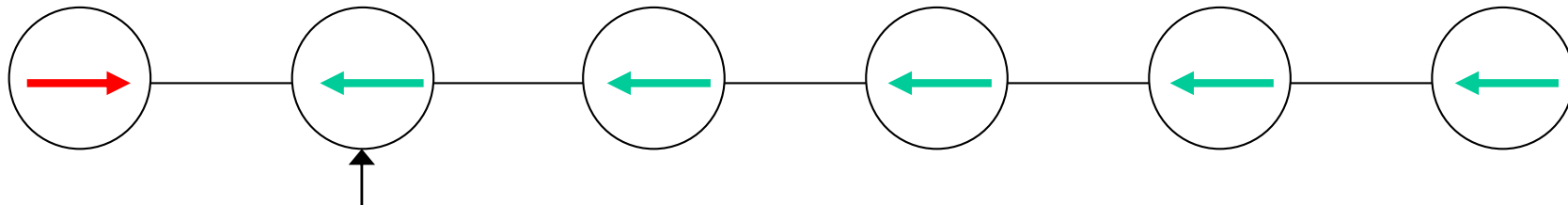
Top



Middle



B



T

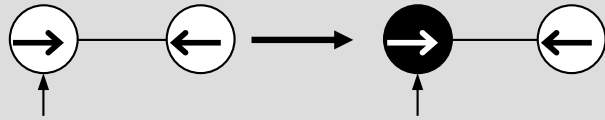
Bottom  

Top  

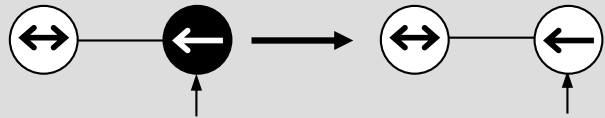
Middle    

 [Dijkstra 74]

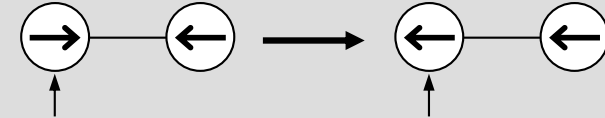
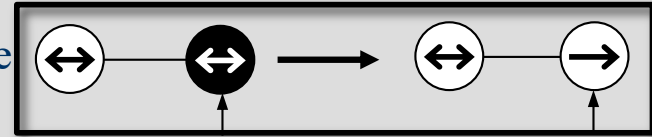
Bottom



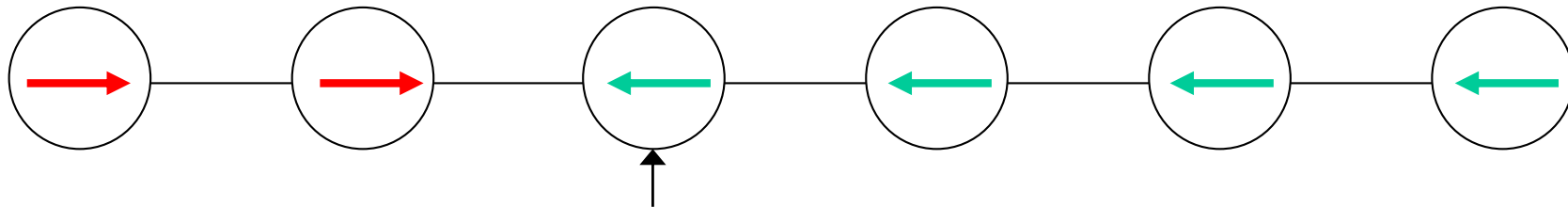
Top



Middle



B



T

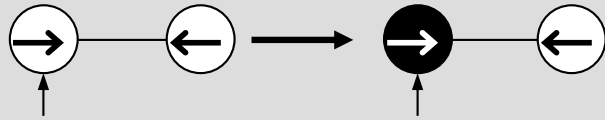
Bottom  

Top  

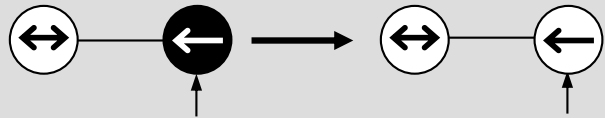
Middle    

 [Dijkstra 74]

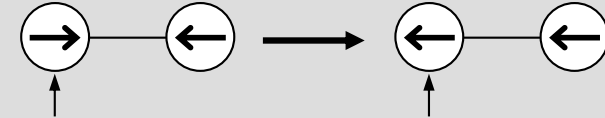
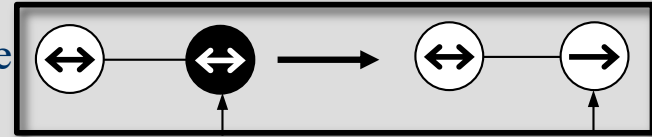
Bottom



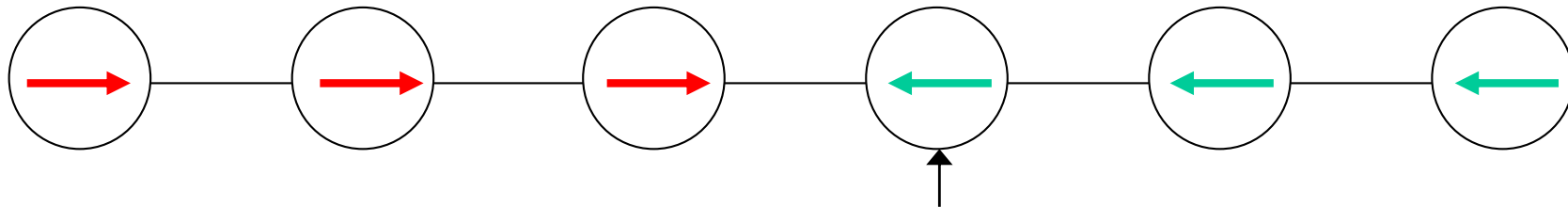
Top



Middle



B



T

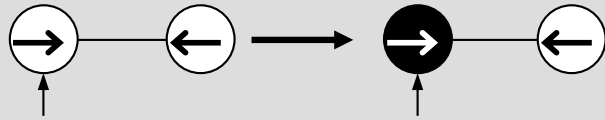
Bottom  

Top  

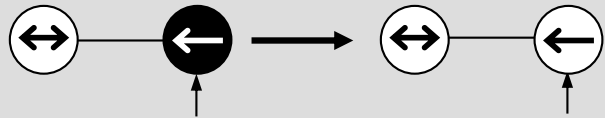
Middle    

 [Dijkstra 74]

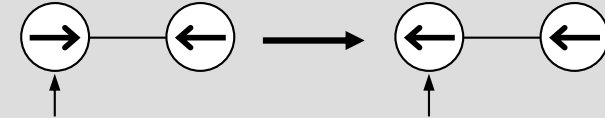
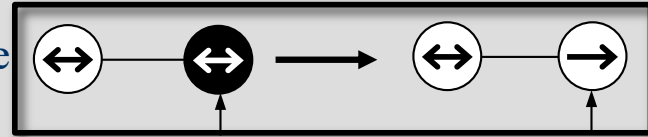
Bottom



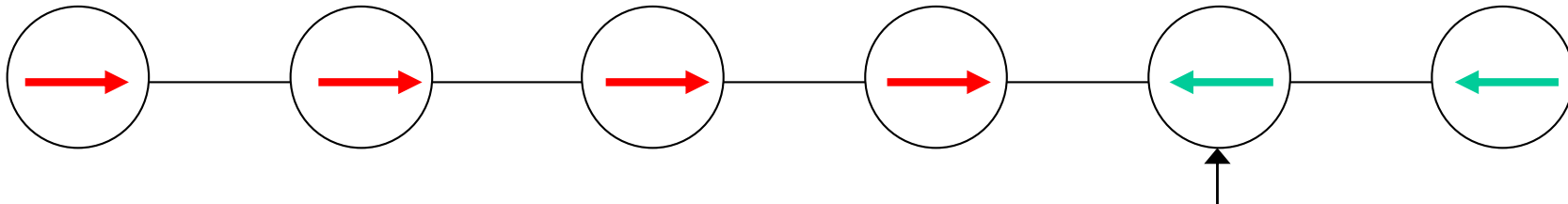
Top



Middle



B



T

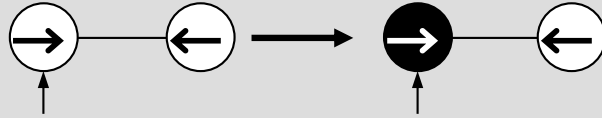
Bottom  

Top  

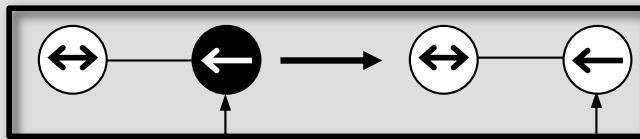
Middle    

 [Dijkstra 74]

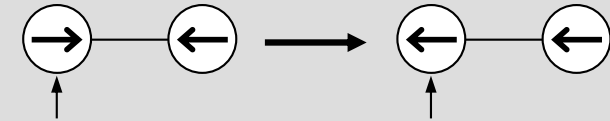
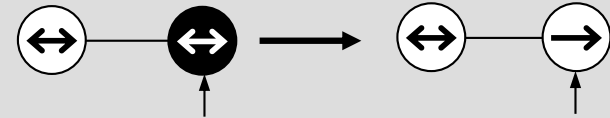
Bottom



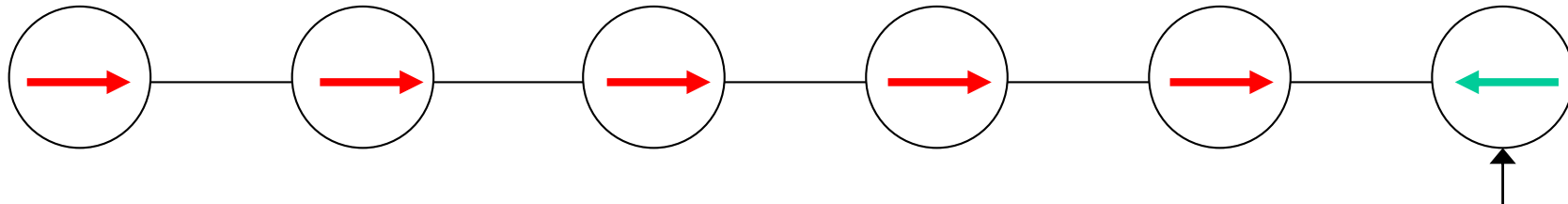
Top



Middle



B



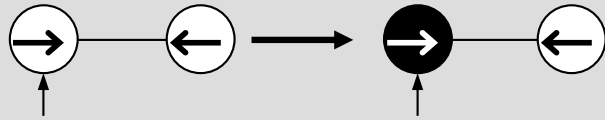
Bottom  

Top  

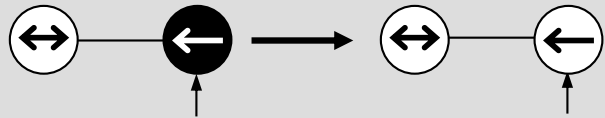
Middle    

 [Dijkstra 74]

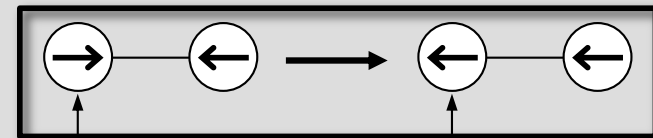
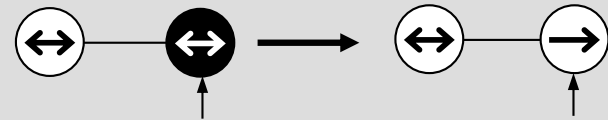
Bottom



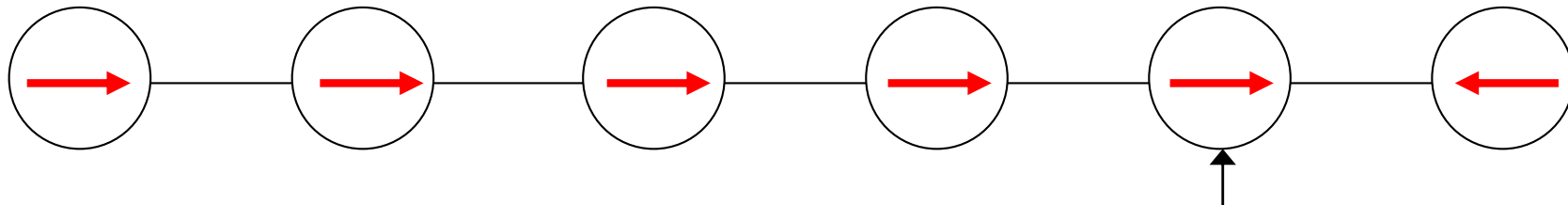
Top



Middle



B



T

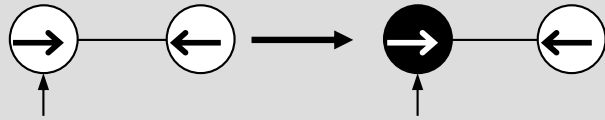
Bottom  

Top  

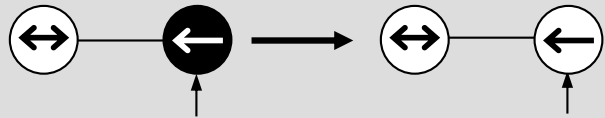
Middle    

 [Dijkstra 74]

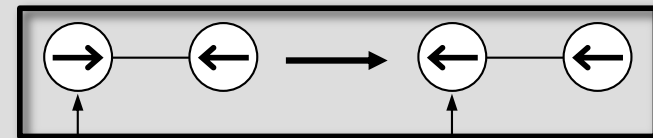
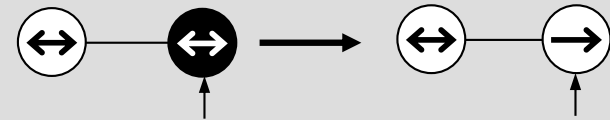
Bottom



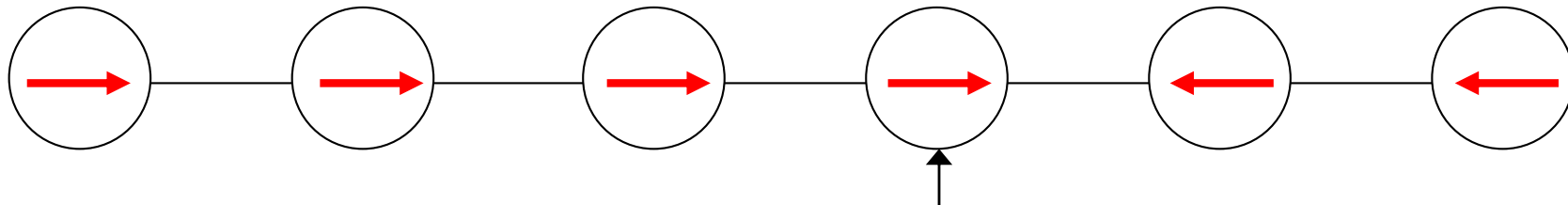
Top



Middle



B



T

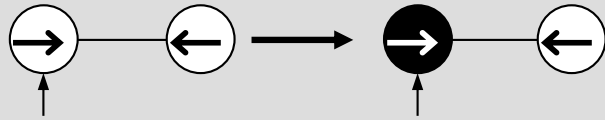
Bottom  

Top  

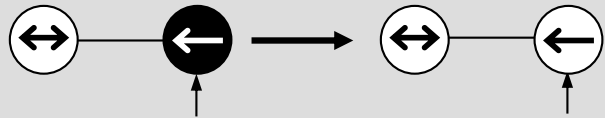
Middle    

 [Dijkstra 74]

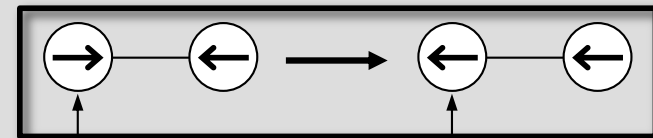
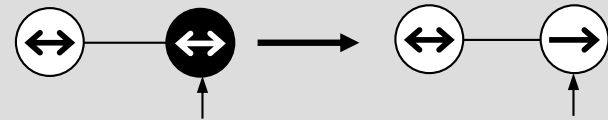
Bottom



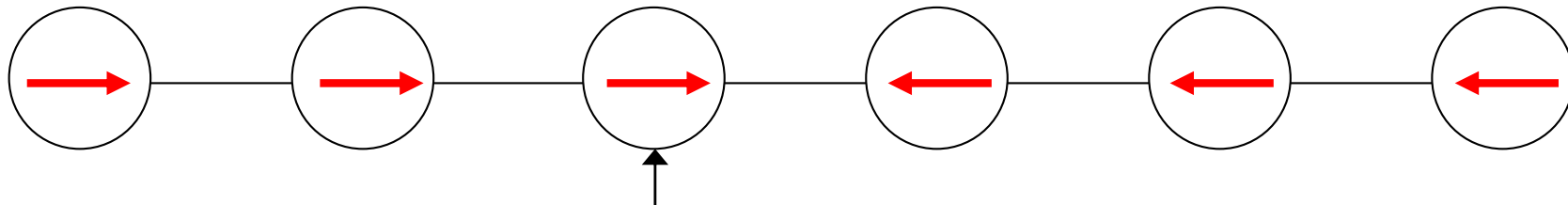
Top



Middle



B



T

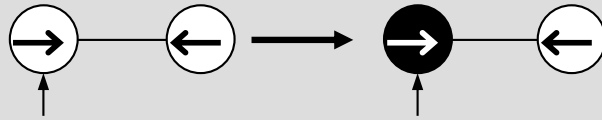
Bottom  

Top  

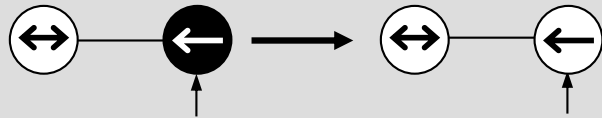
Middle    

 [Dijkstra 74]

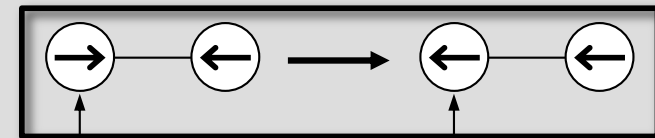
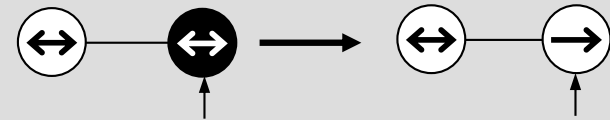
Bottom



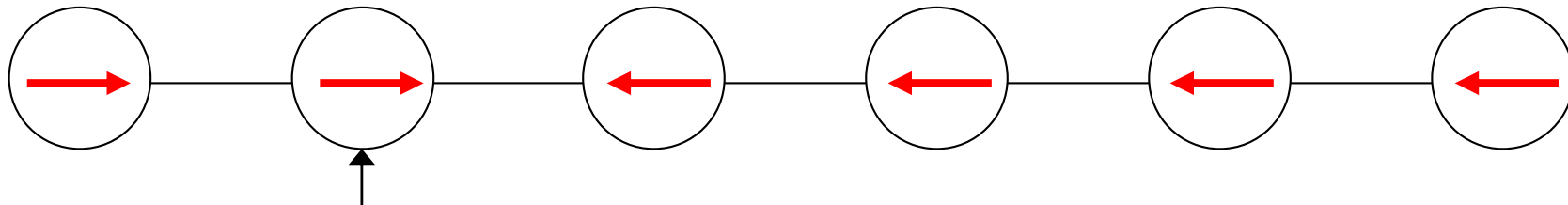
Top



Middle



B



T

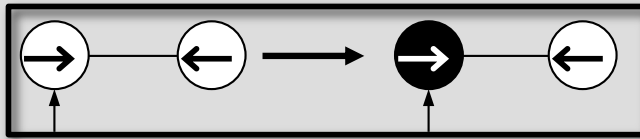
Bottom  

Top  

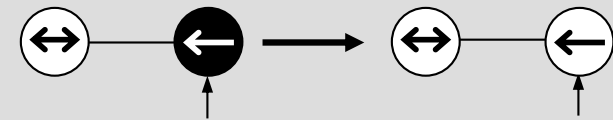
Middle    

 [Dijkstra 74]

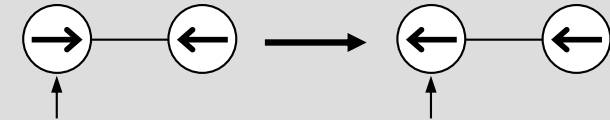
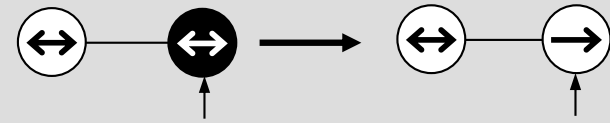
Bottom



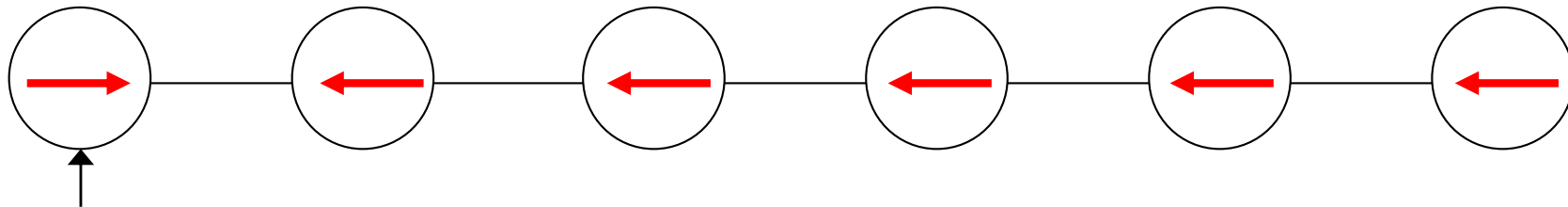
Top



Middle



B



T

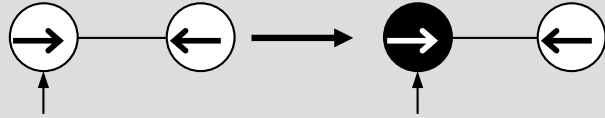
Bottom  

Top  

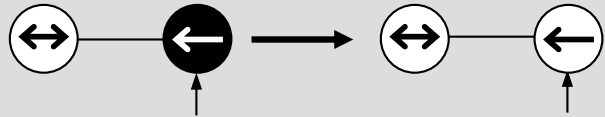
Middle    

 [Dijkstra 74]

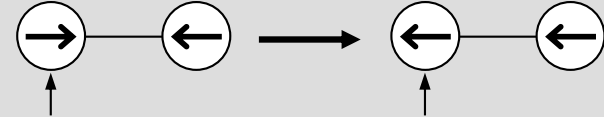
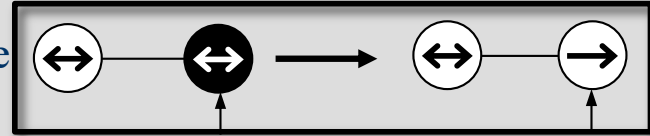
Bottom



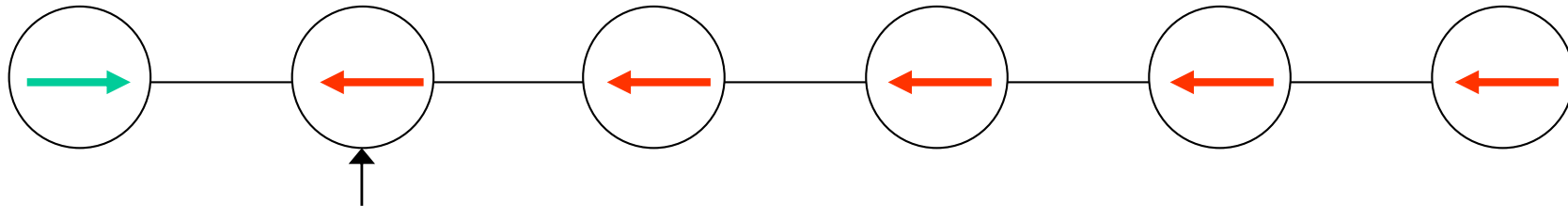
Top



Middle



B



T



Auto-stabilisation ?

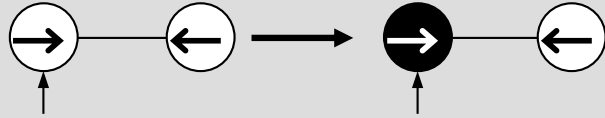
Bottom  

Top  

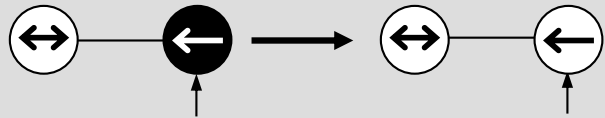
Middle    

 [Dijkstra 74]

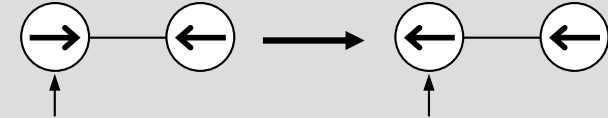
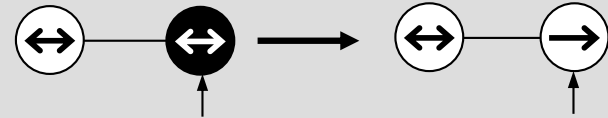
Bottom



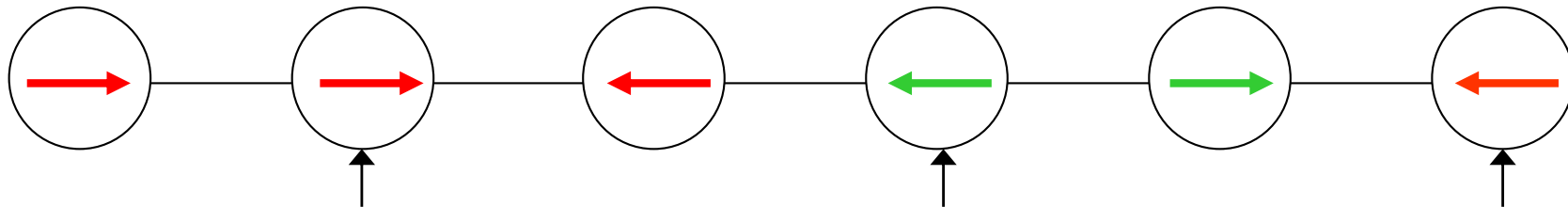
Top



Middle



B



T

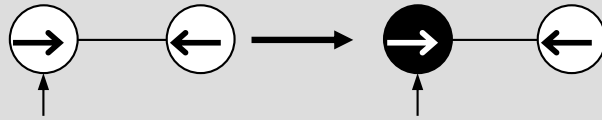
Bottom  

Top  

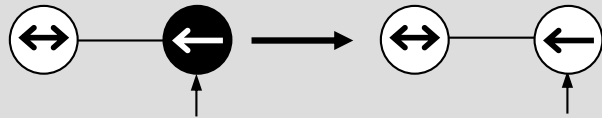
Middle    

 [Dijkstra 74]

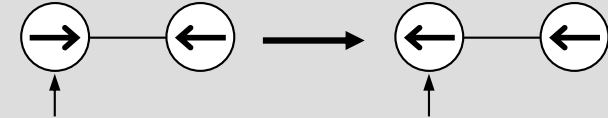
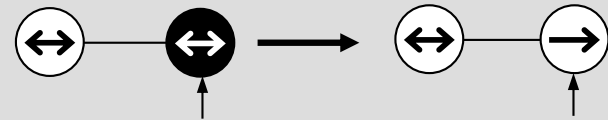
Bottom



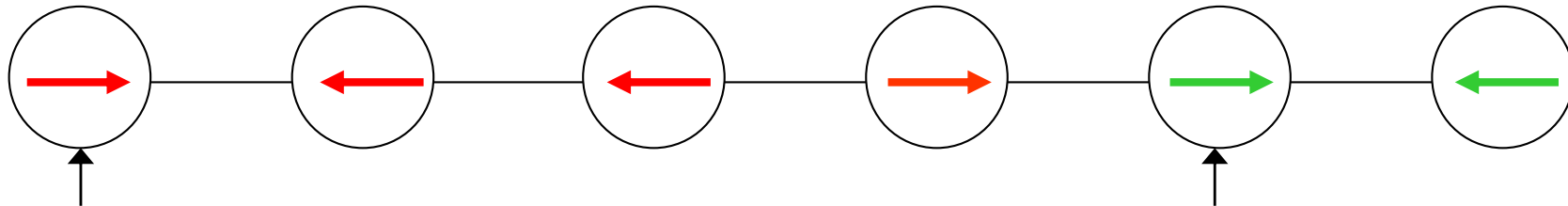
Top



Middle



B



T

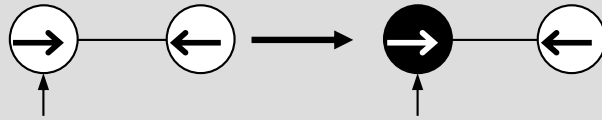
Bottom  

Top  

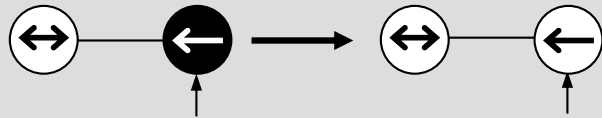
Middle    

 [Dijkstra 74]

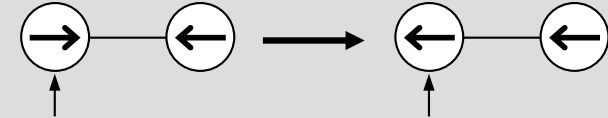
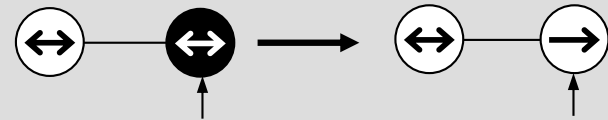
Bottom



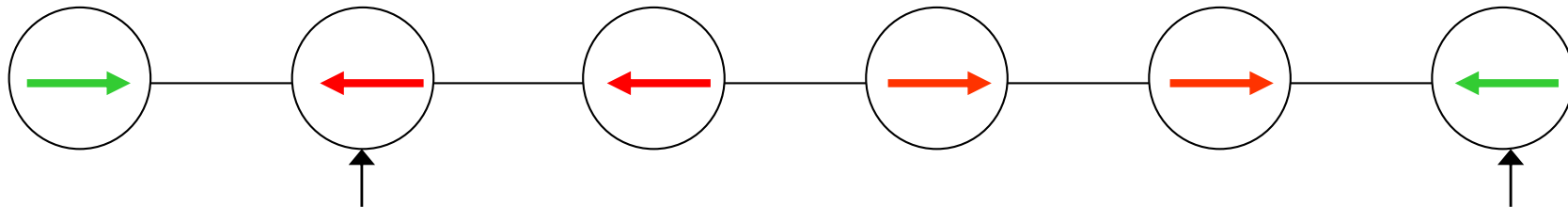
Top



Middle



B



T

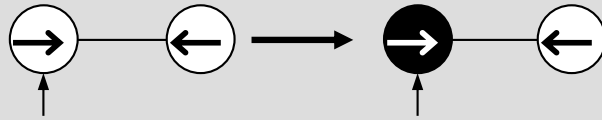
Bottom  

Top  

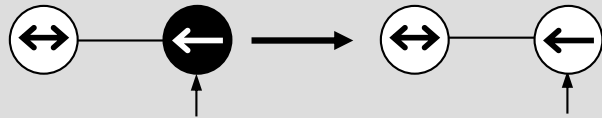
Middle    

 [Dijkstra 74]

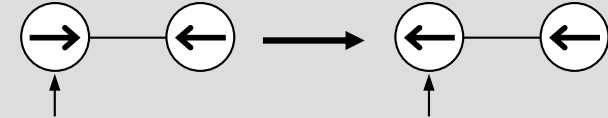
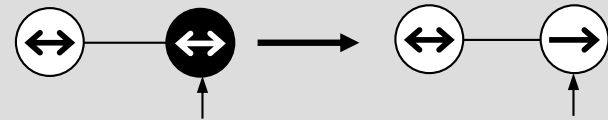
Bottom



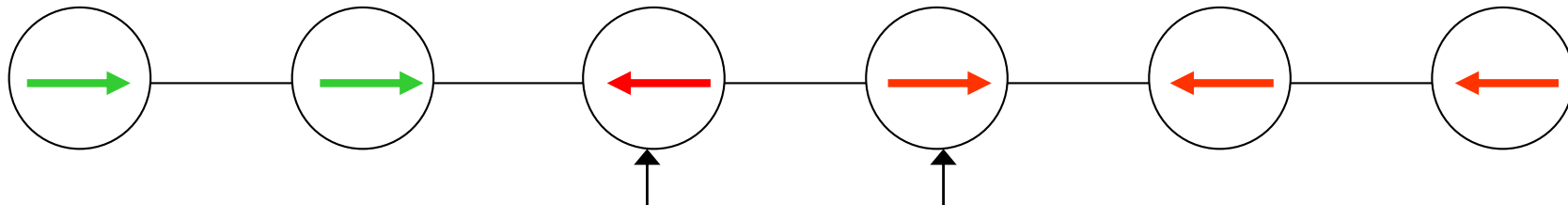
Top



Middle



B



T

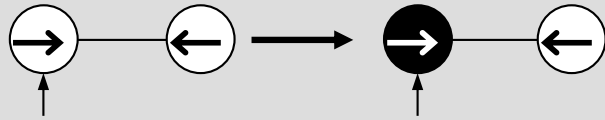
Bottom  

Top  

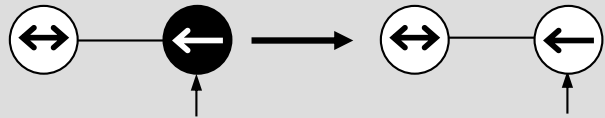
Middle    

 [Dijkstra 74]

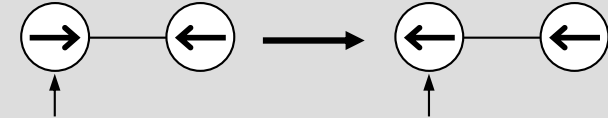
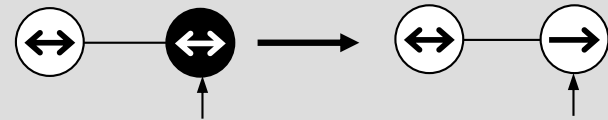
Bottom



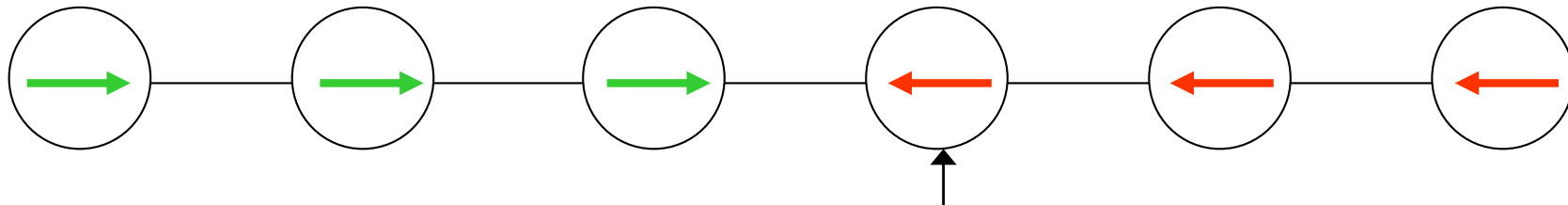
Top



Middle



B



T

Stabilisé !

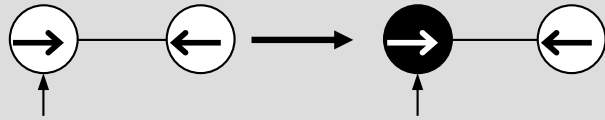
Bottom  

Top  

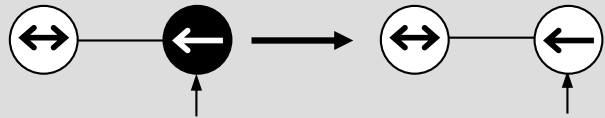
Middle    

 [Dijkstra 74]

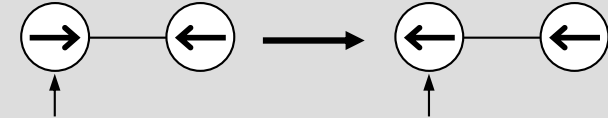
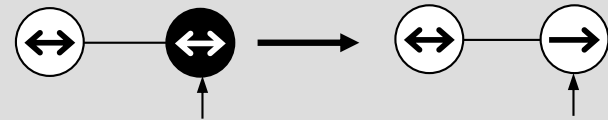
Bottom



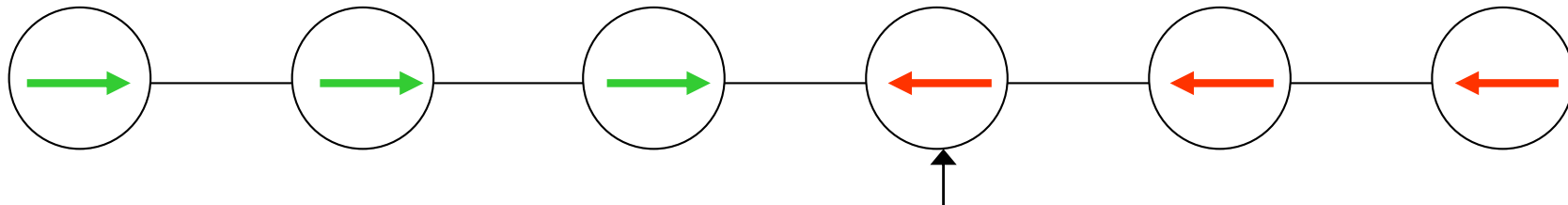
Top



Middle



B



T

Stabilisé !

**Temps de stabilisation =
 $O(n)$**

Self-stabilization

oooooooooooooooo

Hypothesis



ooooo
ooooo

Composition



ooooo

Proof Techniques



oo
o

Conclusion

ooo

References



Self-stabilization

Hypothesis

Atomicity

Scheduling

Composition

Fair Composition

Proof Techniques

Transfer Function

Convergence stairs

Conclusion

References

Atomicity

- ▶ An example of a “stabilizing” sequential program

```
int x = 0;  
  
...  
while( x == x ) {  
    x = 0;  
    // code assuming x equals 0  
}
```

Atomicity

► An example of a “stabilizing” sequential program

```
int x = 0;
...
while( x == x ) {
    x = 0;
    // code assuming x equals 0
}
```

```
0  iconst_0
1  istore_1
2  goto 7
...
5  iconst_0
6  istore_1
7  iload_1
8  iload_1
9  if_icmpeq 5
```


Atomicity

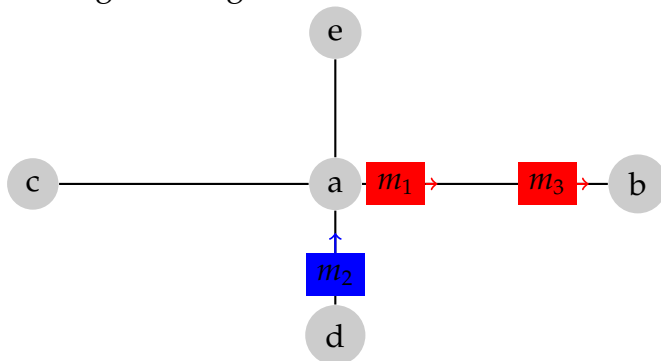
► An example of a “stabilizing” sequential program

```
int x = 0;
...
while( x == x ) {
    x = 0;
    // code assuming x equals 0
}
```

```
0  iconst_0
1  istore_1
2  goto 7
...
5  iconst_0
6  istore_1
7  iload_1
8  iload_1
9  if_icmpeq 5
```

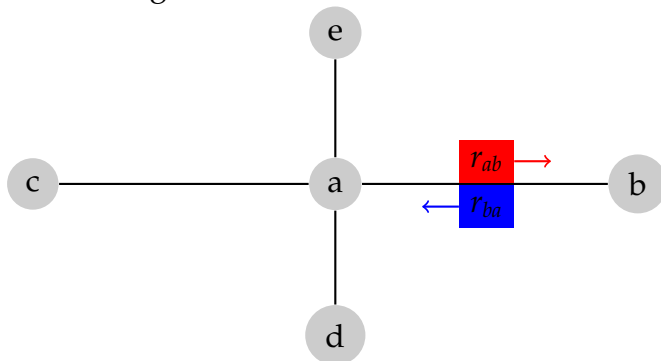
Communications

► Message Passing



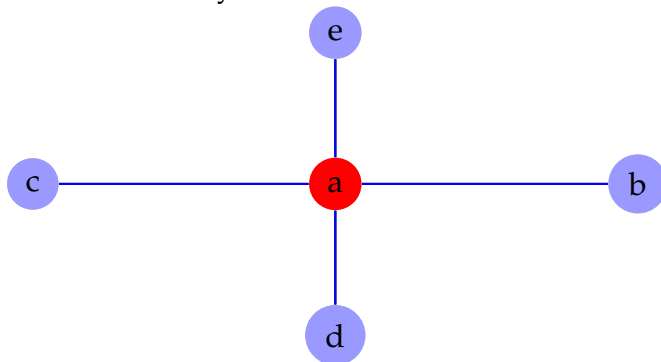
Communications

► Shared Registers



Communications

► Shared Memory



Communications

Message Passing

Shared Registers

Shared Memory

Communications

is more
difficult
to program
than

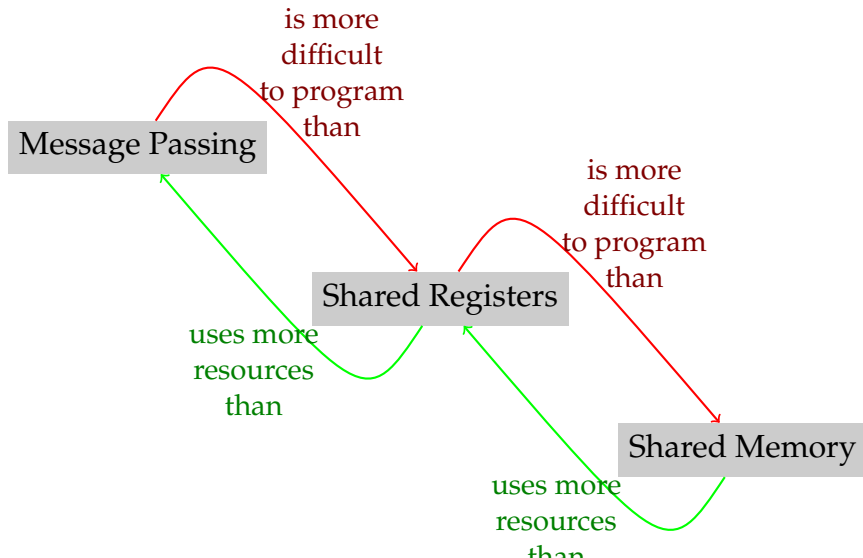
Message Passing

is more
difficult
to program
than

Shared Registers

Shared Memory

Communications



Example

Definition (Shared Memory)

In one atomic step, read the states of all neighbors and write own state

Definition (Guarded command)

► Guard $\rightarrow$ Action

Example

Definition (Shared Memory)

In one atomic step, read the states of all neighbors and write own state

Definition (Guarded command)

- ▶ **Guard** $\rightarrow$ Action
- ▶ Guard: predicate on the states of the neighborhood

Example

Definition (Shared Memory)

In one atomic step, read the states of all neighbors and write own state

Definition (Guarded command)

- ▶ Guard $\rightarrow$ Action
- ▶ Guard: predicate on the states of the neighborhood
- ▶ Action: executed if *Guard* evaluates to true

Self-stabilization
oooooooooooooooo

Hypothesis
o
oooo●
ooooo

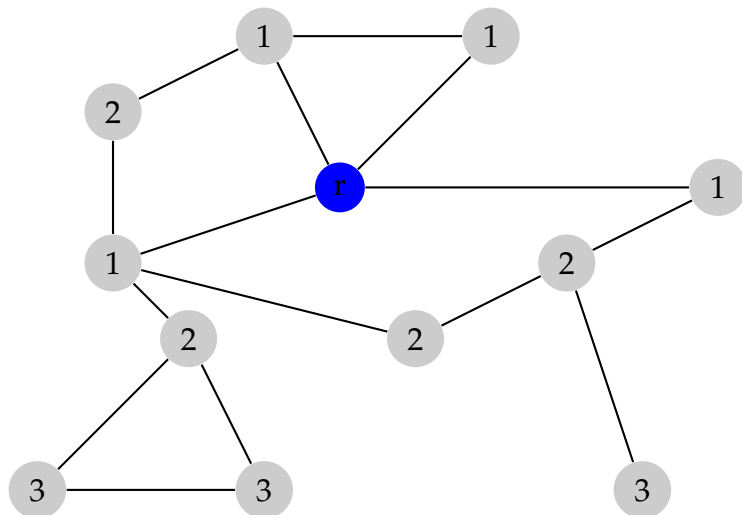
Composition
o
ooooo

Proof Techniques
o
oo
o

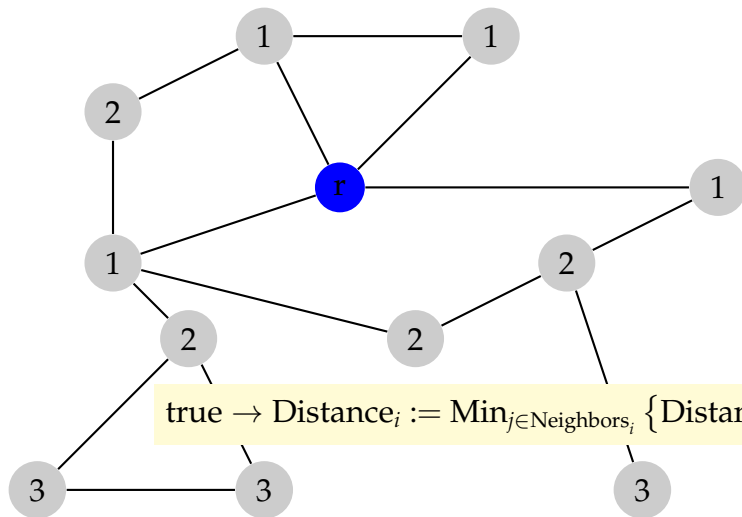
Conclusion
ooo

References
o

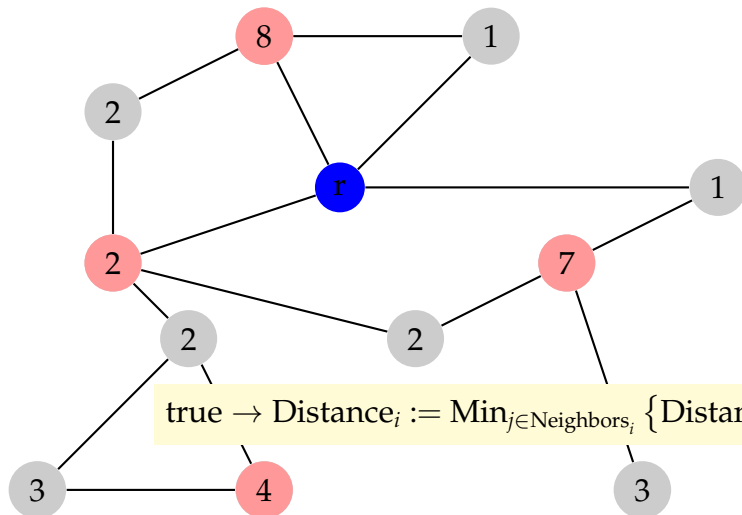
Example



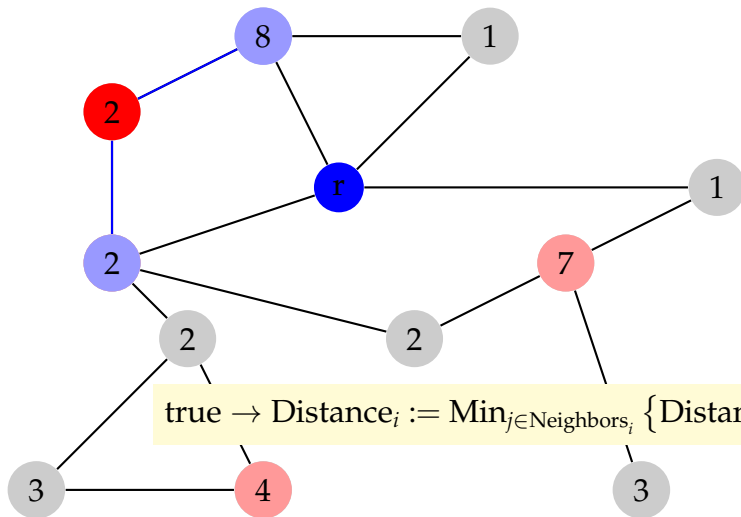
Example



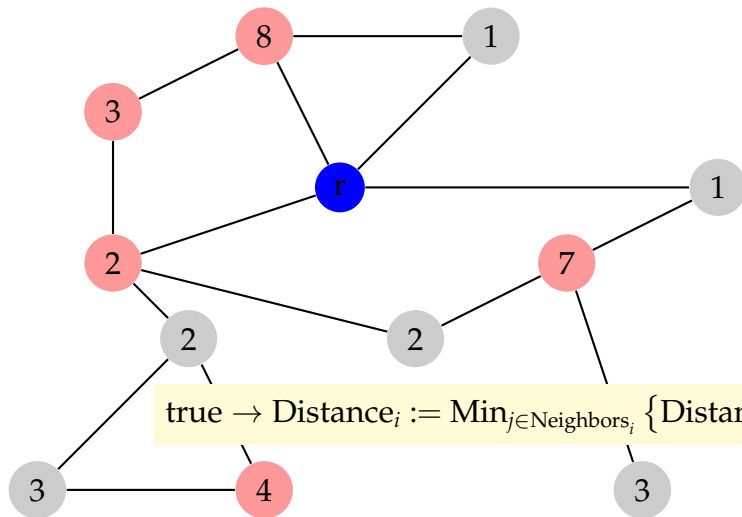
Example



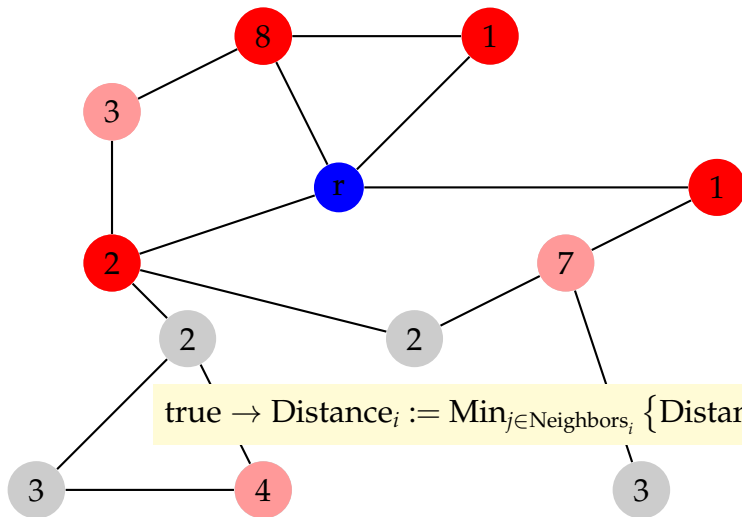
Example



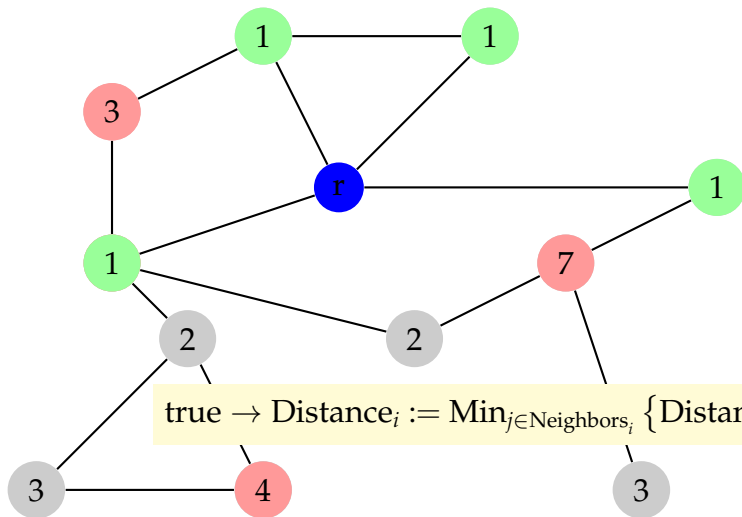
Example



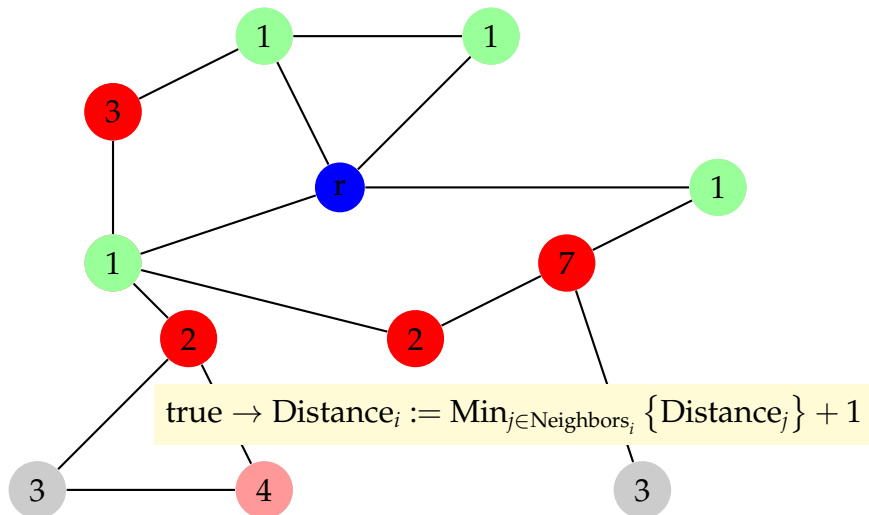
Example



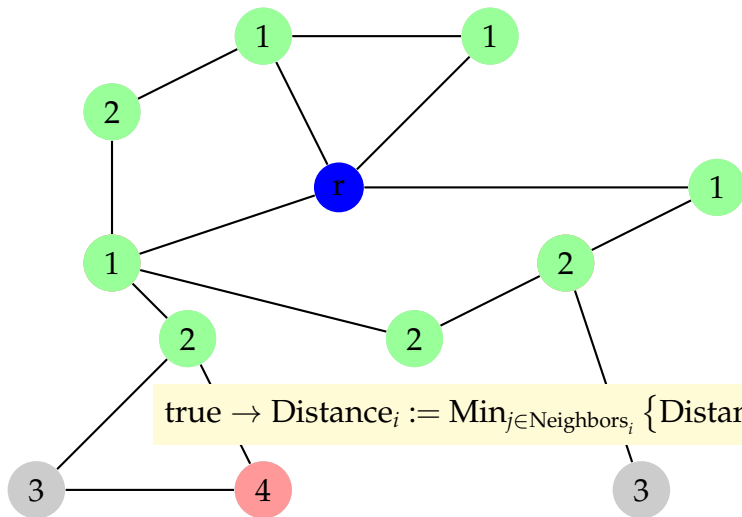
Example



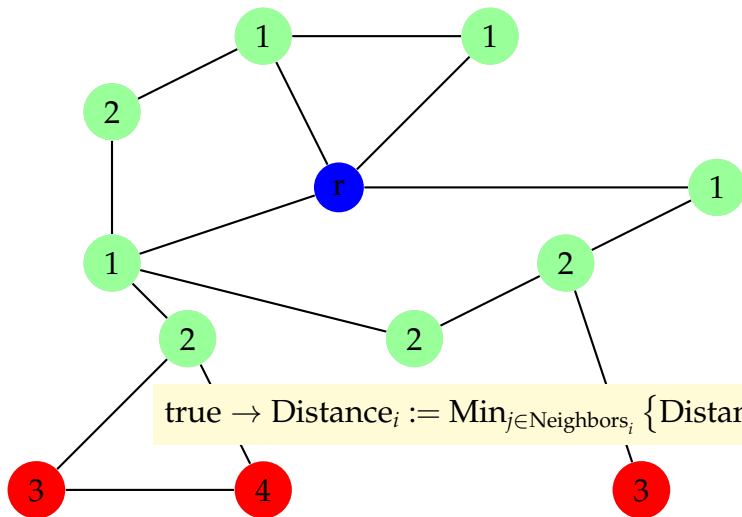
Example



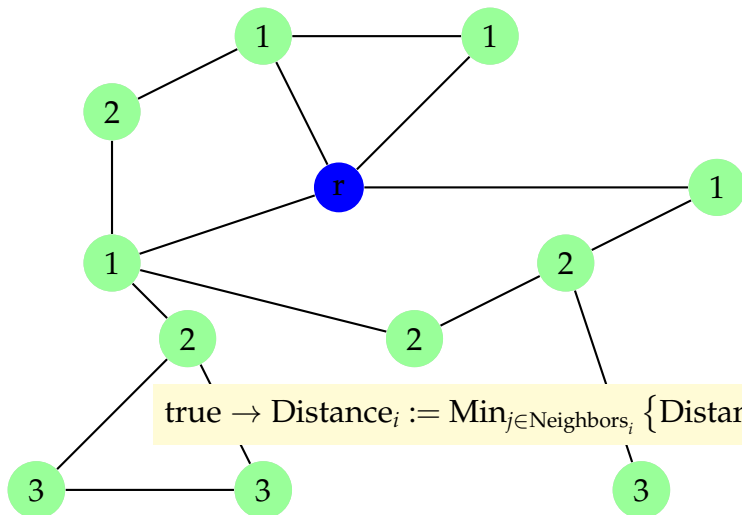
Example



Example



Example



Scheduling

Definition (Scheduler *a.k.a.* Daemon)

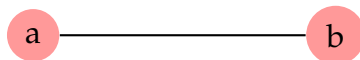
The daemon chooses among activable processors those that will execute their actions.

- ▶ The **daemon** can be seen as an adversary whose role is to prevent stabilization

Spatial Scheduling

$\text{true} \rightarrow \text{color}_i := \text{Min} \{ \Delta \setminus \{ \text{color}_j \mid j \in \text{Neighbors}_i \} \}$

$$\Delta = \{ \text{0}, \text{1} \}$$



Spatial Scheduling

$$\text{true} \rightarrow \text{color}_i := \text{Min} \{ \Delta \setminus \{ \text{color}_j \mid j \in \text{Neighbors}_i \} \}$$

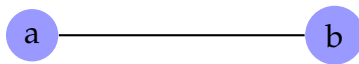
$$\Delta = \{ \text{0}, \text{1} \}$$



Spatial Scheduling

$$\text{true} \rightarrow \text{color}_i := \text{Min} \{ \Delta \setminus \{ \text{color}_j \mid j \in \text{Neighbors}_i \} \}$$

$$\Delta = \{ \textcolor{red}{0}, \textcolor{blue}{1} \}$$



Spatial Scheduling

$\text{true} \rightarrow \text{color}_i := \text{Min} \{ \Delta \setminus \{ \text{color}_j \mid j \in \text{Neighbors}_i \} \}$

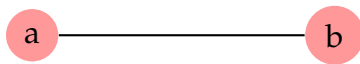
$$\Delta = \{ \text{0}, \text{1} \}$$



Spatial Scheduling

$$\text{true} \rightarrow \text{color}_i := \text{Min} \{ \Delta \setminus \{ \text{color}_j \mid j \in \text{Neighbors}_i \} \}$$

$$\Delta = \{ \textcolor{red}{0}, \textcolor{blue}{1} \}$$



Spatial Scheduling

$$\text{true} \rightarrow \text{color}_i := \text{Min} \{ \Delta \setminus \{ \text{color}_j \mid j \in \text{Neighbors}_i \} \}$$

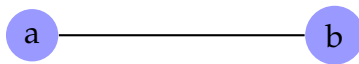
$$\Delta = \{ \textcolor{red}{0}, \textcolor{blue}{1} \}$$



Spatial Scheduling

$$\text{true} \rightarrow \text{color}_i := \text{Min} \{ \Delta \setminus \{ \text{color}_j \mid j \in \text{Neighbors}_i \} \}$$

$$\Delta = \{ \textcolor{red}{0}, \textcolor{blue}{1} \}$$



Spatial Scheduling

$$\text{true} \rightarrow \text{color}_i := \text{Min} \{ \Delta \setminus \{ \text{color}_j \mid j \in \text{Neighbors}_i \} \}$$

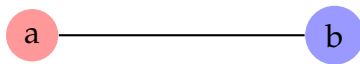
$$\Delta = \{ \textcolor{red}{0}, \textcolor{blue}{1} \}$$



Spatial Scheduling

$$\text{true} \rightarrow \text{color}_i := \text{Min} \{ \Delta \setminus \{ \text{color}_j \mid j \in \text{Neighbors}_i \} \}$$

$$\Delta = \{ \textcolor{red}{0}, \textcolor{blue}{1} \}$$



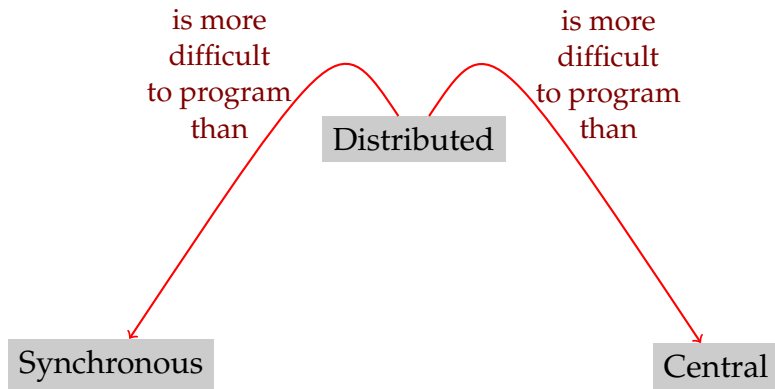
Spatial Scheduling

Distributed

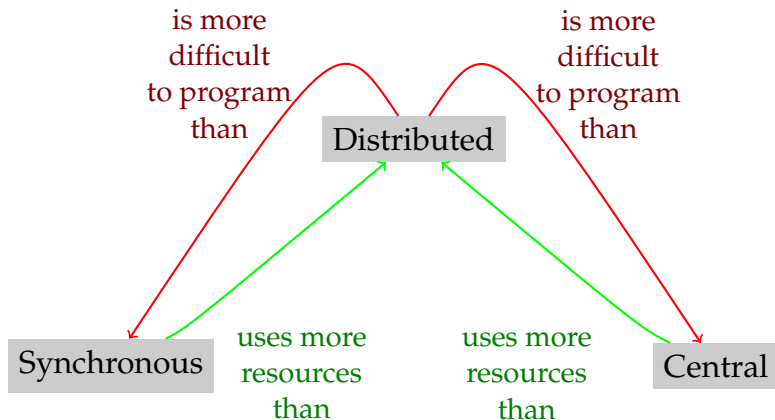
Synchronous

Central

Spatial Scheduling

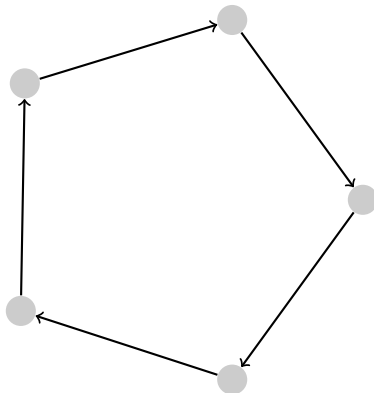


Spatial Scheduling



Temporal Scheduling

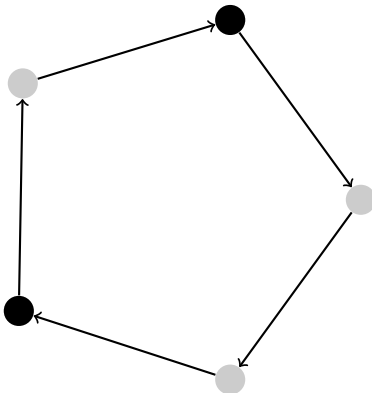
token $\rightarrow$ pass token to left neighbor with probability $\frac{1}{2}$



● = token

Temporal Scheduling

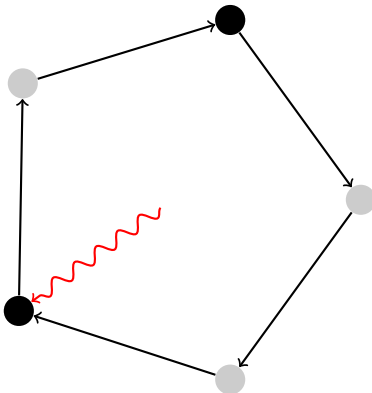
token $\rightarrow$ pass token to left neighbor with probability $\frac{1}{2}$



● = token

Temporal Scheduling

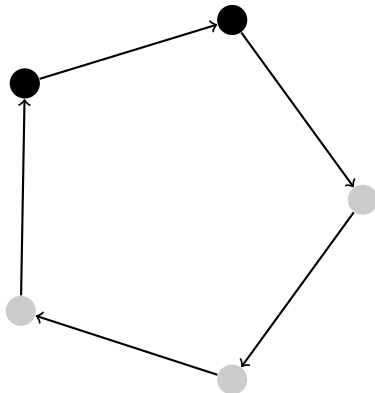
token $\rightarrow$ pass token to left neighbor with probability $\frac{1}{2}$



● = token

Temporal Scheduling

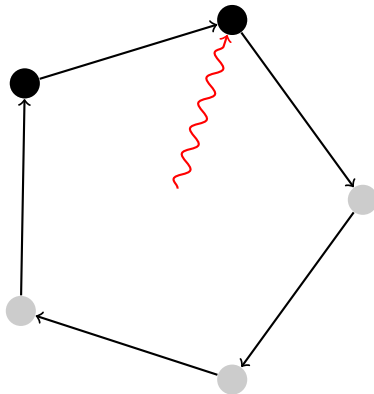
token $\rightarrow$ pass token to left neighbor with probability $\frac{1}{2}$



● = token

Temporal Scheduling

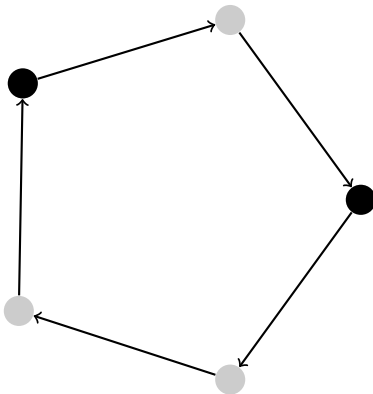
token $\rightarrow$ pass token to left neighbor with probability $\frac{1}{2}$



● = token

Temporal Scheduling

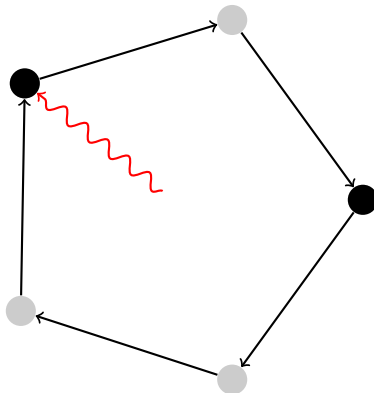
token $\rightarrow$ pass token to left neighbor with probability $\frac{1}{2}$



● = token

Temporal Scheduling

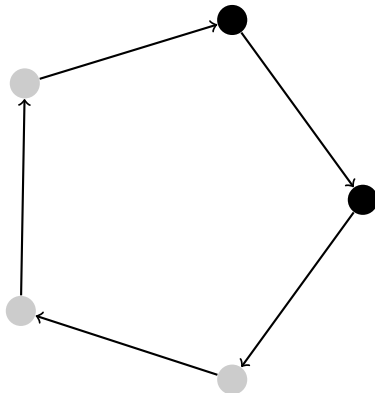
token $\rightarrow$ pass token to left neighbor with probability $\frac{1}{2}$



● = token

Temporal Scheduling

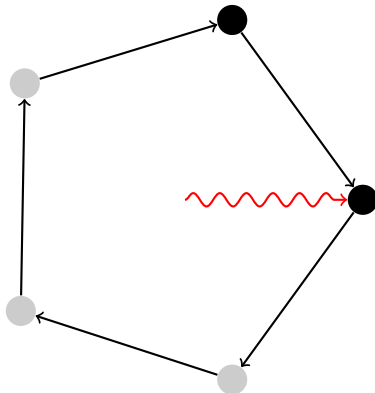
token $\rightarrow$ pass token to left neighbor with probability $\frac{1}{2}$



● = token

Temporal Scheduling

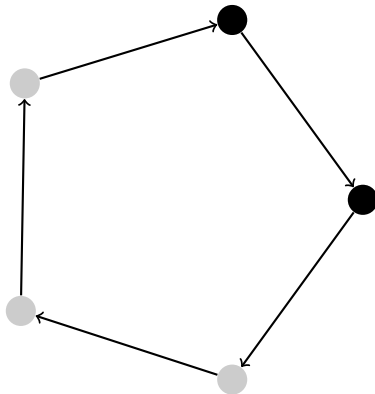
token $\rightarrow$ pass token to left neighbor with probability $\frac{1}{2}$



● = token

Temporal Scheduling

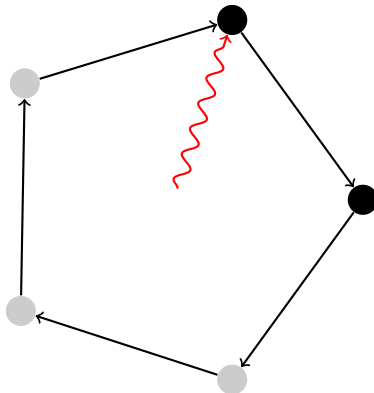
token $\rightarrow$ pass token to left neighbor with probability $\frac{1}{2}$



● = token

Temporal Scheduling

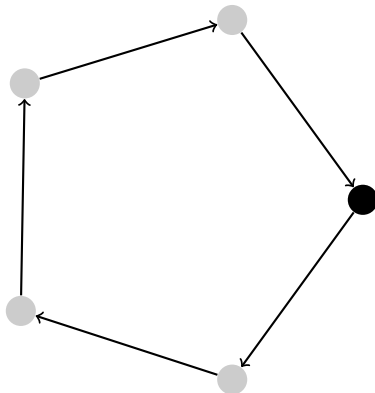
token $\rightarrow$ pass token to left neighbor with probability $\frac{1}{2}$



● = token

Temporal Scheduling

token $\rightarrow$ pass token to left neighbor with probability $\frac{1}{2}$



● = token

Temporal Scheduling

Unfair

Fair

Bounded

Temporal Scheduling

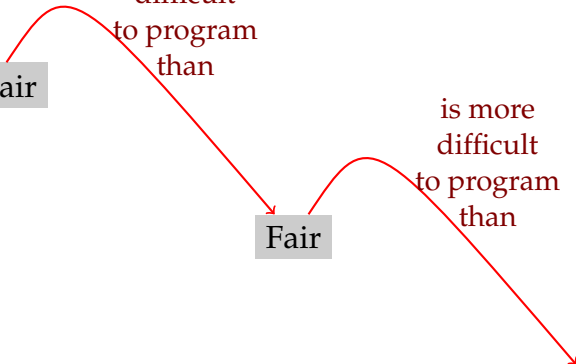
is more
difficult
to program
than

Unfair

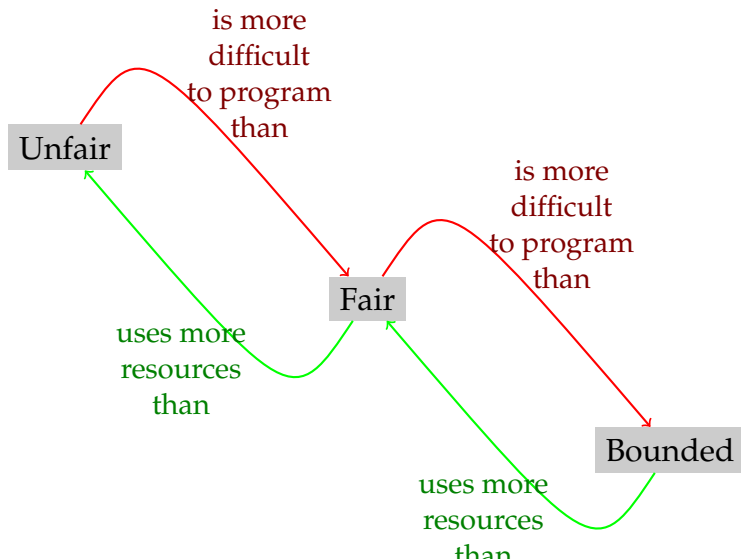
is more
difficult
to program
than

Fair

Bounded



Temporal Scheduling



Self-stabilization

oooooooooooooooo

Hypothesis

o
ooooo
ooooo

Composition

●
ooooo

Proof Techniques

o
oo
o

Conclusion

ooo

References

o

Self-stabilization

Hypothesis

Atomicity

Scheduling

Composition

Fair Composition

Proof Techniques

Transfer Function

Convergence stairs

Conclusion

References

Fair Composition

Basic idea

- ▶ Compose several self-stabilizing algorithms $Al_1, Al_2, \dots, Al_k$ such that the results of algorithms $Al_1, Al_2, \dots, Al_i$ can be reused by Al_{i+1}
- ▶ Al_{i+1} can not detect whether algorithms $Al_1, Al_2, \dots, Al_i$ have stabilized, but behaves as if

Fair Composition

Basic idea

- ▶ Compose several self-stabilizing algorithms $Al_1, Al_2, \dots, Al_k$ such that the results of algorithms $Al_1, Al_2, \dots, Al_i$ can be reused by Al_{i+1}
- ▶ Al_{i+1} can not detect whether algorithms $Al_1, Al_2, \dots, Al_i$ have stabilized, but behaves as if

Example with $k = 2$

- ▶ Two simple algorithms server and client are combined to obtain a more complex algorithm
- ▶ The server algorithm ensures that some properties (used by the client) will be eventually verified

Example

- Assume the server algorithm Al_1 solves a task defined by a set of legal executions T_1 , and the client algorithm Al_2 solves T_2

A_i

- Let A_i be the set of states of process P_i for Al_1 , and let $S_i = A_i \times B_i$ be the set of states of process P_i for Al_2 , where anytime P_i executes Al_2 , it modifies the B_i part of $A_i \times B_i$

$A_i - B_i$

Fair Composition

Definition (Fair composition)

Al is a **fair composition** of Al_1 and Al_2 if, in Al , every process alternatively executes actions of Al_1 and Al_2

Theorem

If Al_2 is self-stabilizing for T_2 given T_1 , and if Al_1 is self-stabilizing for T_1 , then the fair composition of Al_1 and Al_2 is self-stabilizing for T_2

$$A_i \rightarrow B_i$$

$$Al_1 \quad Al_2$$

Fair Composition

Definition (Fair composition)

Al is a **fair composition** of Al_1 and Al_2 if, in Al , every process alternatively executes actions of Al_1 and Al_2

Theorem

If Al_2 is self-stabilizing for T_2 given T_1 , and if Al_1 is self-stabilizing for T_1 , then the fair composition of Al_1 and Al_2 is self-stabilizing for T_2

$$A_i \rightarrow B_i$$

$$Al_1 \quad Al_2$$

Fair Composition

Definition (Fair composition)

Al is a **fair composition** of Al_1 and Al_2 if, in Al , every process alternatively executes actions of Al_1 and Al_2

Theorem

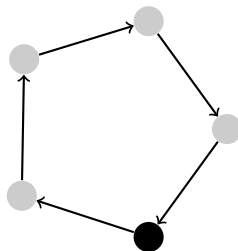
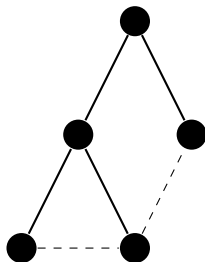
If Al_2 is self-stabilizing for T_2 given T_1 , and if Al_1 is self-stabilizing for T_1 , then the fair composition of Al_1 and Al_2 is self-stabilizing for T_2

$$A_i \rightarrow B_i$$

$$Al_1 \quad Al_2$$

Example

- ▶ We are given two self-stabilizing algorithms, one for constructing a tree, one for mutual exclusion on a ring
- ▶ We wish to construct a self-stabilizing mutual exclusion algorithm on general graphs



Self-stabilization
oooooooooooooooo

Hypothesis
○
○○○○
○○○○

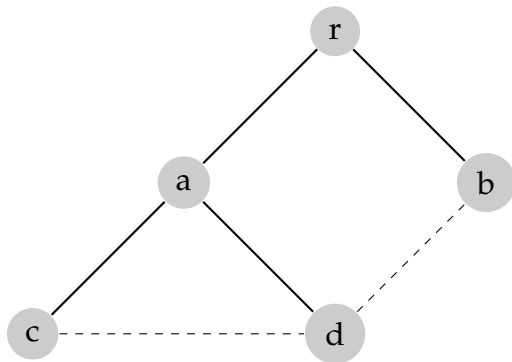
Composition
○
○○○○●

Proof Techniques
○
○○
○

Conclusion
○○○

References
○

Example



Self-stabilization
oooooooooooooooo

Hypothesis
○
○○○○
○○○○

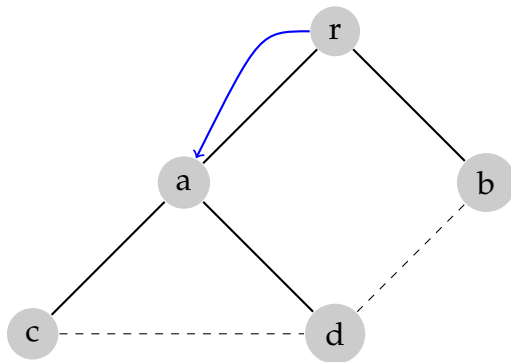
Composition
○
○○○○●

Proof Techniques
○
○○
○

Conclusion
○○○

References
○

Example



Self-stabilization
oooooooooooooooo

Hypothesis
○
○○○○
○○○○

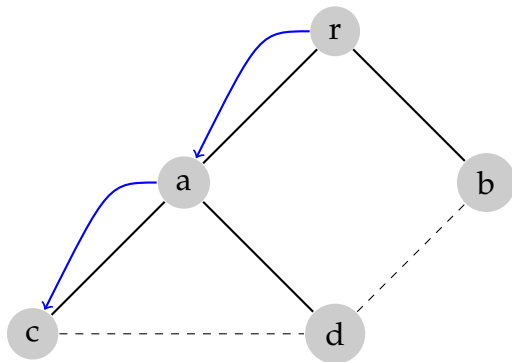
Composition
○
○○○○●

Proof Techniques
○
○○
○

Conclusion
○○○

References
○

Example



Self-stabilization
oooooooooooooooo

Hypothesis
○
○○○○
○○○○

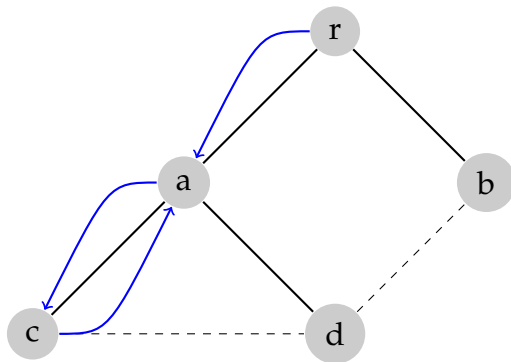
Composition
○
○○○○●

Proof Techniques
○
○○
○

Conclusion
○○○

References
○

Example



Self-stabilization
oooooooooooooooo

Hypothesis
○
○○○○
○○○○

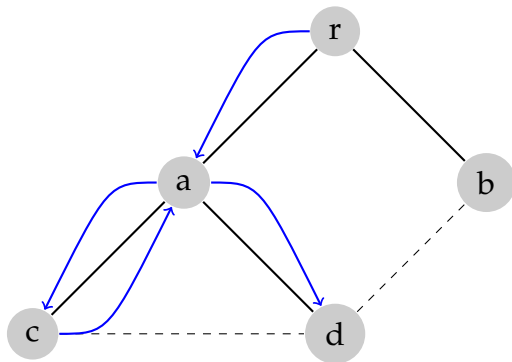
Composition
○
○○○○●

Proof Techniques
○
○○
○

Conclusion
○○○

References
○

Example



Self-stabilization
oooooooooooooooo

Hypothesis
○
○○○○
○○○○

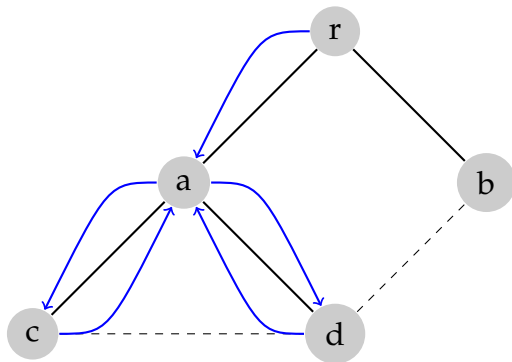
Composition
○
○○○○●

Proof Techniques
○
○○
○

Conclusion
○○○

References
○

Example



Self-stabilization
oooooooooooooooo

Hypothesis
○
○○○○
○○○○

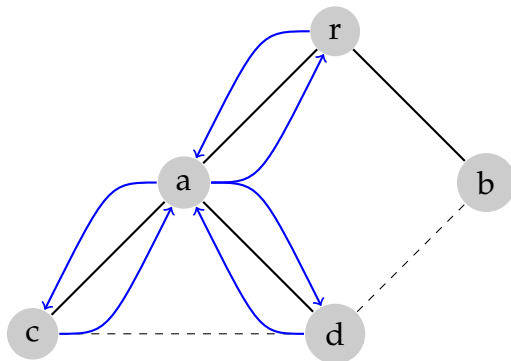
Composition
○
○○○○●

Proof Techniques
○
○○
○

Conclusion
○○○

References
○

Example



Self-stabilization
oooooooooooooooo

Hypothesis
○
○○○○
○○○○

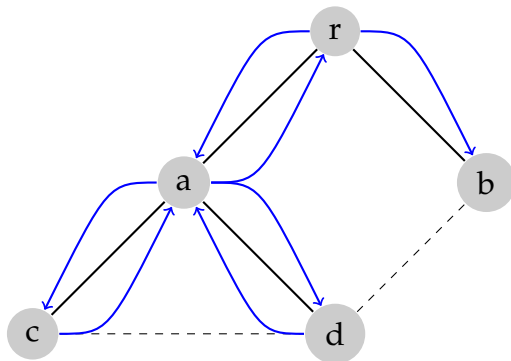
Composition
○
○○○○●

Proof Techniques
○
○○
○

Conclusion
○○○

References
○

Example



Self-stabilization
oooooooooooooooo

Hypothesis
○
○○○○
○○○○

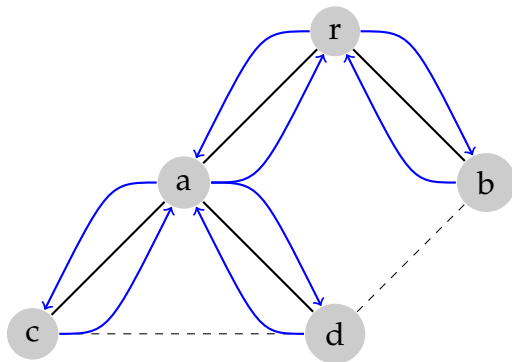
Composition
○
○○○○●

Proof Techniques
○
○○
○

Conclusion
○○○

References
○

Example



Self-stabilization

oooooooooooooooo

Hypothesis

o
oooo
oooo

Composition

o
oooo

Proof Techniques

●
oo
o

Conclusion

ooo

References

o

Self-stabilization

Hypothesis

Atomicity

Scheduling

Composition

Fair Composition

Proof Techniques

Transfer Function

Convergence stairs

Conclusion

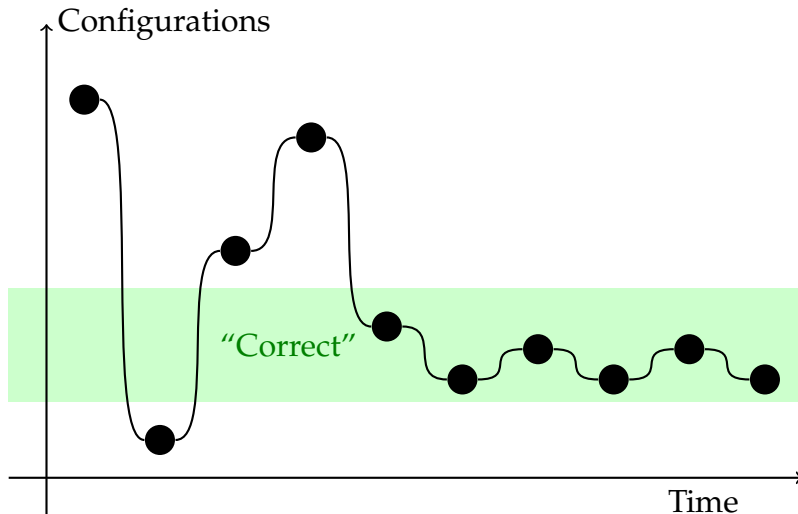
References

Transfer Function

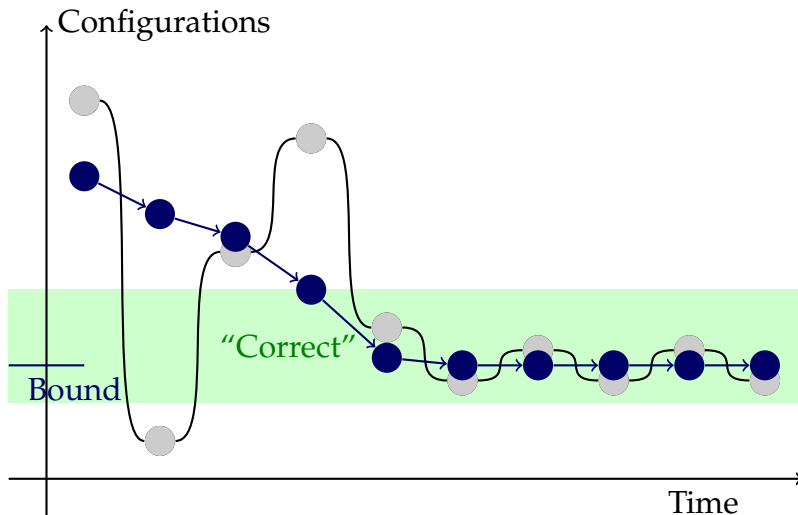
Basic Idea

- ▶ $c_1 \rightarrow c_2 \rightarrow c_3 \rightarrow c_4 \rightarrow \dots \rightarrow c_i$
- ▶ $FP(c_1) > FP(c_2) > FP(c_3) > \dots > FP(c_i) = \text{bound}$
- ▶ Used to prove convergence
- ▶ Can be used to compute the number of steps to reach a legitimate configuration

Transfer Function

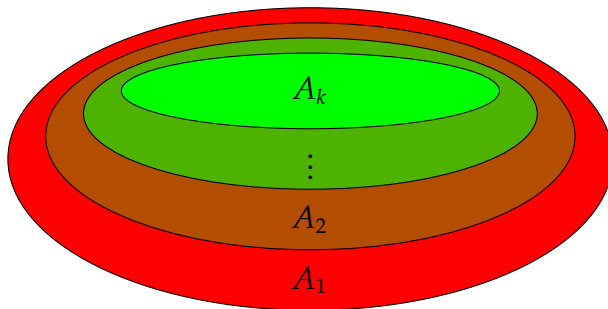


Transfer Function



Convergence stairs

- ▶ A_i is a predicate
- ▶ A_k is legitimate
- ▶ For any i between 1 and k , A_{i+1} is a refinement of A_i



Self-stabilization

oooooooooooooooo

Hypothesis

o
ooooo
ooooo

Composition

o
ooooo

Proof Techniques

o
oo
o

Conclusion

●oo

References

o

Self-stabilization

Hypothesis

Atomicity

Scheduling

Composition

Fair Composition

Proof Techniques

Transfer Function

Convergence stairs

Conclusion

References

Self-stabilization

Pros

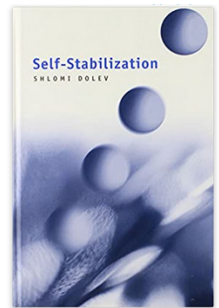
- ▶ The network does not need to be initialized
- ▶ When a fault is diagnosed, it is sufficient to identify, then remove or restart the faulty components
- ▶ The self-stabilization property does not depend on the nature of the fault
- ▶ The self-stabilization property does not depend on the extent of the fault

Self-stabilization

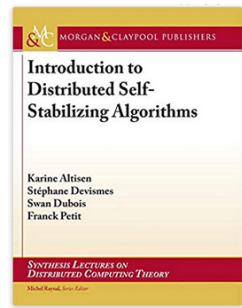
Cons

- ▶ *A priori*, “eventually” does not give any bound on the stabilization time
- ▶ *A priori*, nodes never know whether the system is stabilized or not
- ▶ A single failure may trigger a correcting action at every node in the network
- ▶ Faults must be sufficiently rare that they can be considered are transient

References



Self-Stabilization.
S. Dolev
The MIT Press, 2000.



Introduction to Distributed Self-Stabilizing Algorithms.
K. Altisen, S. Devismes, S. Dubois, and F. Petit
Morgan & Claypool Publishers, 2019.